

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

"Diseño Táctico del Dominio para la Gestión Integral de Servicios de Mecánica Automotriz"

Asignatura:

Ingeniería de Software III

Docente: Roger Manuel Sánchez Chávez

Alumno:

• Limay Rodriguez, Adriana Anthonela (100%)
• Pérez Briceño, Darick Andre (100%)
• Llanos Cerquin, Melanie Brizeth (100%)
• Valdiviezo Zavaleta, Jesus Arturo (100%)
• Villanueva Mendoza, Rosselly (100%)

Cajamarca - Perú
Agosto, 2026

ÍNDICE

1. Introducción..... 5

2. Descripción de Entidades..... 5

2.1. Contexto Acotado: Recepción y Diagnóstico (Core Domain)..... 5

2.1.1. Entidad Vehículo..... 6

2.1.2. Entidad Cliente..... 6

2.1.3. Entidad RecepcionVehicular..... 7

2.1.4. Entidad InspeccionFisica..... 8

2.1.5. Entidad DiagnosticoOBD2..... 9

2.1.6. Entidad Cotizacion..... 9

2.2. Contexto Acotado: Operaciones de Taller (Core Domain)..... 10

2.2.1. Entidad OrdenTrabajo..... 10

2.2.2. Entidad TareaReparacion..... 11

2.2.3. Entidad BahiaTrabajo..... 12

2.3. Contexto Acotado: Control de Calidad y Entrega (Core Domain)..... 13

2.3.1. Entidad PruebaCalidad..... 13

2.3.2. Entidad InformeEntrega..... 14

2.4. Contexto Acotado: Inventario y Almacén (Supporting Subdomain)..... 15

2.4.1. Entidad Repuesto..... 15

2.4.2. Entidad ReservaRepuesto..... 16

2.5. Contexto Acotado: Agendamiento (Supporting Subdomain)..... 16

2.5.1. Entidad Cita..... 16

2.6. Contexto Acotado: Gestión de Personal (Supporting Subdomain)..... 17

2.6.1. Entidad Tecnico..... 17

2.7. Contexto Acotado: Facturación y Finanzas (Generic Subdomain)..... 18

2.7.1. Entidad ComprobantePago..... 18

3. Descripción de Objetos de valor..... 19

3.1. Contexto Acotado: Recepción y Diagnóstico..... 19

3.1.1. Value Object Placa..... 19

3.1.2. Value Object Kilometraje..... 20

3.1.3. Value Object CodigoFalla..... 20

3.1.4. Value Object ItemCotizacion..... 20

3.1.5. Value Object AutorizacionCliente..... 21

3.1.6. Value Object Monto..... 21

3.1.7. Value Object ContactoCliente..... 22

3.1.8. Value Objects de Identidad..... 22

3.2. Contexto Acotado: Operaciones de Taller..... 25

3.2.1. Value Object FotoEvidencia..... 25

3.2.2. Value Object RegistroUsoRepuesto..... 26

3.2.3. Value Objects de Identidad..... 26

3.3. Contexto Acotado: Control de Calidad y Entrega..... 29

3.3.1. Value Object CondicionGarantia..... 29

3.3.2. Value Object RegistroMantenimientoSugerido..... 29

3.3.3. Value Object CheckSeguridad..... 29

3.3.4. Value Objects de Identidad..... 30

3.4. Contexto Acotado: Inventario y Almacén.....32
3.4.1. Value Object UmbralReposicion.....32
3.4.2. Value Object Monto.....32
3.4.3. Value Objects de Identidad.....32
3.5. Contexto Acotado: Agendamiento.....33
3.5.1. Value Object SlotHorario.....33
3.5.2. Value Objects de Identidad.....33
3.6. Contexto Acotado: Gestión de Personal.....34
3.6.1. Value Object TurnoHorario.....34
3.6.2. Value Objects de Identidad.....34
3.7. Contexto Acotado: Facturación y Finanzas.....35
3.7.1. Value Object Monto.....35
3.7.2. Value Objects de Identidad.....35
4. Descripción de Agregados.....36
4.1. CONTEXTO ACOTADO: RECEPCIÓN Y DIAGNÓSTICO.....36
4.1.1. AGREGADO 1: Cliente.....36
4.1.1. AGREGADO 2: RecepcionVehicular.....39
4.1.2. AGREGADO 3: Vehiculo.....44
4.1.3. AGREGADO 4: Cotizacion.....48
4.2. CONTEXTO ACOTADO: OPERACIONES DE TALLER.....51
4.2.1. AGREGADO 5: OrdenTrabajo.....51
4.2.2. AGREGADO 6: BahiaTrabajo.....56
4.3. CONTEXTO ACOTADO: CONTROL DE CALIDAD Y ENTREGA.....59
4.3.1. AGREGADO 7: PruebaCalidad.....59
4.3.2. AGREGADO 8: InformeEntrega.....62
4.4. CONTEXTO ACOTADO: INVENTARIO Y ALMACÉN.....65
4.4.3. AGREGADO 9: Repuesto.....65
4.4.4. AGREGADO 10: ReservaRepuesto.....69
4.5. CONTEXTO ACOTADO: AGENDAMIENTO.....72
4.5.5. AGREGADO 11: Cita.....72
4.6. CONTEXTO ACOTADO: GESTIÓN DE PERSONAL.....76
4.6.6. AGREGADO 12: Tecnico.....76
4.7. CONTEXTO ACOTADO: FACTURACIÓN Y FINANZAS.....80
4.7.1. AGREGADO 13: ComprobantePago.....80
4.8. Diagrama de Agregados.....84
5. Servicios de Dominio.....85
5.1. Servicio de Aprobación y Abastecimiento (ServicioAprobacionCotizacion).....85
5.2. Servicio de Orquestación de Bahías (ServicioAsignacionTaller).....85
5.3. Servicio de Evaluación de Garantías (ServicioEvaluacionGarantia).....86
6. Eventos de Dominio.....87
6.1. Contexto Acotado: Recepción y Diagnóstico.....87
6.1.1. Agregado: Vehiculo.....87
6.1.2. Agregado: RecepcionVehicular.....87
6.1.3. Agregado: Cotizacion.....87
6.2. Contexto Acotado: Operaciones de Taller.....87
6.2.1. Agregado: OrdenTrabajo.....87

6.2.2. Agregado: BahiaTrabajo..... 88
6.3. Contexto Acotado: Control de Calidad y Entrega..... 88
6.3.1. Agregado: PruebaCalidad..... 88
6.3.2. Agregado: InformeEntrega..... 88
6.4. Contexto Acotado: Inventario y Almacén..... 88
6.4.1. Agregado: Repuesto..... 88
6.4.2. Agregado: ReservaRepuesto..... 89
6.5. Contexto Acotado:Agendamiento..... 89
6.5.1. Agregado: Cita..... 89
6.6. Contexto Acotado: Gestión de Personal..... 89
6.6.1. Agregado: Tecnico..... 89
6.7. Contexto Acotado: Facturación y Finanzas..... 89
6.7.1. Agregado: ComprobantePago..... 89
7. Conclusiones..... 91
8. Referencias..... 92

1. Introducción

El presente informe documenta el Diseño Táctico del Dominio aplicado al sistema de Gestión Integral de Servicios de Mecánica Automotriz Cajamarca (MAC), desarrollado bajo los principios y patrones de Domain-Driven Design (DDD). Este trabajo constituye la continuación natural del diseño estratégico previamente elaborado, donde se identificaron los contextos acotados (*Bounded Contexts*), el lenguaje ubicuo y el mapa de contexto del dominio.

El objetivo central de este diseño táctico es traducir el conocimiento del negocio, capturado durante la fase estratégica, en un modelo de dominio rico, expresivo y técnicamente implementable. Para ello, se han identificado y modelado 16 entidades, 13 agregados, múltiples objetos de valor (*Value Objects*), 3 servicios de dominio y un conjunto completo de eventos de dominio (*Domain Events*), distribuidos a lo largo de 7 contextos acotados que abarcan la totalidad del ciclo operativo del taller:

- Recepción y Diagnóstico (*Core Domain*): Gestiona el ingreso del vehículo, la inspección física, el diagnóstico OBD2 y la elaboración de cotizaciones con autorización explícita del cliente.
- Operaciones de Taller (*Core Domain*): Coordina las órdenes de trabajo, la asignación de bahías y técnicos, y el seguimiento de las tareas de reparación con evidencia fotográfica.
- Control de Calidad y Entrega (*Core Domain*): Supervisa las pruebas estáticas y de ruta, genera el informe de entrega y programa el mantenimiento predictivo.
- Inventario y Almacén (*Supporting Subdomain*): Administra el catálogo de repuestos, el control de stock físico, las reservas preventivas y los umbrales de reposición automática.
- Agendamiento (*Supporting Subdomain*): Gestiona las citas y la programación de turnos de atención.
- Gestión de Personal (*Supporting Subdomain*): Controla la disponibilidad, turnos y especialidades de los técnicos del taller.
- Facturación y Finanzas (*Generic Subdomain*): Emite y administra los comprobantes de pago conforme a la normativa tributaria de SUNAT.

El diseño presta especial atención a la resolución de problemáticas operativas críticas detectadas en el análisis del dominio, tales como el fenómeno del "stock fantasma" (situación en la que el inventario lógico del sistema no coincide con la existencia física real de repuestos) y la necesidad de garantizar la trazabilidad completa del servicio desde la recepción hasta la entrega del vehículo. Los servicios de dominio diseñados (*ServicioAprobacionCotizacion*, *ServicioAsignacionTaller* y *ServicioEvaluacionGarantia*) orquestan operaciones que cruzan múltiples agregados, manteniendo la coherencia transaccional y el cumplimiento de las reglas de negocio sin generar un modelo anémico.

Cada entidad y agregado ha sido implementado en Java, respetando los principios de encapsulamiento, inmutabilidad de identidades y protección de invariantes de negocio. Los objetos de valor encapsulan conceptos semánticos del dominio (como Monto, CondicionGarantia, SlotHorario, UmbralReposicion, entre otros), dotando al modelo de un lenguaje ubicuo que facilita la comunicación entre el equipo de desarrollo y los expertos del dominio.

2. Descripción de Entidades

Las entidades representan los conceptos clave del taller que tienen una identidad única y que persistente a lo largo del tiempo, dentro de cada entidad, los atributos de tipo identificador (ej. VehiculoId, RecepcionId) actúan como Value Objects de Identidad para garantizar tipado fuerte y evitar errores en código.

2.1. Contexto Acotado: Recepción y Diagnóstico (Core Domain)

Representa el automotor que ingresa al taller, su identidad persiste a lo largo de múltiples visitas.

2.1.1. Entidad Vehículo

Atributos:

- VehiculoId id (Value Object de Identidad)
- Placa placa (Value Object)
- String vin (Número de Identificación Vehicular)
- String marca
- String modelo
- Kilometraje kilometrajeActual (Value Object: Kilometraje)
- ClienteId clienteId (Value Object de Identidad)

Métodos:

- actualizarKilometraje(Kilometraje nuevoKilometraje): Valida que el nuevo kilometraje no sea inferior al actual delegando la comparación al Value Object.

Código:

```
public class Vehiculo {
   private final VehiculoId id;
   private Placa placa;
   private String vin;
   private String marca;
   private String modelo;
   private Kilometraje kilometrajeActual;
   private ClienteId clienteId;

   public Vehiculo(VehiculoId id, Placa placa, String vin, String
marca, String modelo,
                   Kilometraje kilometrajeActual, ClienteId clienteId)
{
       this.id = id;
       this.placa = placa;
       this.vin = vin;
       this.marca = marca;
       this.modelo = modelo;
       this.kilometrajeActual = kilometrajeActual;
       this.clienteId = clienteId;
   }

   public void actualizarKilometraje(Kilometraje nuevoKilometraje) {
       if (nuevoKilometraje.esMenorQue(this.kilometrajeActual)) {
           throw new IllegalArgumentException("El kilometraje no puede
ser menor al registrado previamente.");
       }
       this.kilometrajeActual = nuevoKilometraje;
   }
}
```

2.1.2. Entidad Cliente

Persona natural o jurídica que solicita los servicios del taller y asume la responsabilidad financiera.

Atributos:

- ClienteId id (Value Object de Identidad)
- String dniRuc
- String nombreCompletoRazonSocial

- ContactoCliente contacto
- String direccion

Métodos:

- actualizarDatosContacto(ContactoCliente nuevoContacto)

Código:

```
public class Cliente {
   private final ClienteId id;
   private String dniRuc;
   private String nombreCompletoRazonSocial;
   private ContactoCliente contacto;
   private String direccion;

   public Cliente(ClienteId id, String dniRuc, String
nombreCompletoRazonSocial,
                  ContactoCliente contacto, String direccion) {
       this.id = id;
       this.dniRuc = dniRuc;
       this.nombreCompletoRazonSocial = nombreCompletoRazonSocial;
       this.contacto = contacto;
       this.direccion = direccion;
   }

   public void actualizarDatosContacto(ContactoCliente nuevoContacto) {
       if (nuevoContacto == null) {
           throw new IllegalArgumentException("Los datos de contacto no
pueden ser nulos.");
       }
       this.contacto = nuevoContacto;
   }
}
```

2.1.3. Entidad RecepcionVehicular

Consolida el proceso de ingreso del vehículo al taller, coordinando la inspección exterior y el escaneo OBD-II.

Atributos:

- RecepcionId id (Value Object de Identidad)
- VehiculoId vehiculoId (Value Object de Identidad)
- ClienteId clienteId (Value Object de Identidad)
- DateTime fechaIngreso
- EstadoRecepcion estado (EN_INSPECCION, DIAGNOSTICADO, COMPLETADO, CANCELADO)

Métodos:

- iniciarInspeccion()
- completarRecepcion()

Código:

```
public class RecepcionVehicular {
   private final RecepcionId id;
   private VehiculoId vehiculoId;
   private ClienteId clienteId;
   private LocalDateTime fechaIngreso;
   private EstadoRecepcion estado;
```

```
    public RecepcionVehicular(RecepcionId id, VehiculoId
vehiculoId, ClienteId clienteId) {
        this.id = id;
        this.vehiculoId = vehiculoId;
        this.clienteId = clienteId;
        this.fechaIngreso = LocalDateTime.now();
        this.estado = EstadoRecepcion.EN_INSPECCION;
    }

    public void iniciarInspeccion() {
        this.estado = EstadoRecepcion.EN_INSPECCION;
    }

    public void completarRecepcion() {
        this.estado = EstadoRecepcion.COMPLETADO;
    }
}
```

2.1.4. Entidad InspeccionFisica

Registra la evaluación de carrocería e inventario exterior realizada en conjunto con el cliente antes de la intervención.

Atributos:

- InspeccionFisicaId id (Value Object de Identidad)
- VehiculoId vehiculoId (Value Object de Identidad)
- DateTime fechaRegistro
- List<String> danosCarroceria (Lista de observaciones/rayones)
- boolean nivelCombustible
- List<String> pertenenciasDejadas
- String firmaClienteUrl

Métodos:

- registrarObservacion(String detalle)
- asociarFirmaCliente(String firmaUrl)

Código:

```
public class InspeccionFisica {
    private final InspeccionFisicaId id;
    private VehiculoId vehiculoId;
    private LocalDateTime fechaRegistro;
    private List<String> danosCarroceria;
    private boolean nivelCombustible;
    private List<String> pertenenciasDejadas;
    private String firmaClienteUrl;

    public InspeccionFisica(InspeccionFisicaId id, VehiculoId
vehiculoId) {
        this.id = id;
        this.vehiculoId = vehiculoId;
        this.fechaRegistro = LocalDateTime.now();
        this.danosCarroceria = new ArrayList<>();
        this.pertenenciasDejadas = new ArrayList<>();
    }

    public void registrarObservacion(String detalle) {
```

```
        this.danosCarroceria.add(detalle);
    }

    public void asociarFirmaCliente(String firmaUrl) {
        this.firmaClienteUrl = firmaUrl;
    }
}
```

2.1.5. Entidad DiagnosticoOBD2

Captura el resultado del escaneo electrónico al puerto OBD-II del vehículo.

Atributos:

- DiagnosticoId id (Value Object de Identidad)
- VehiculoId vehiculoId (Value Object de Identidad)
- DateTime fechaEscaneo
- List<CodigoFalla> codigosCapturados (Value Object de Datos: CodigoFalla)
- String notasTecnico

Métodos:

- agregarCodigoFalla(CodigoFalla codigo)
- tieneFallasCriticas(): boolean

Código:

```
public class DiagnosticoOBD2 {
    private final DiagnosticoId id;
    private VehiculoId vehiculoId;
    private LocalDateTime fechaEscaneo;
    private List<CodigoFalla> codigosCapturados; // Value Object de
Datos
    private String notasTecnico;

    public DiagnosticoOBD2(DiagnosticoId id, VehiculoId vehiculoId) {
        this.id = id;
        this.vehiculoId = vehiculoId;
        this.fechaEscaneo = LocalDateTime.now();
        this.codigosCapturados = new ArrayList<>();
    }

    public void agregarCodigoFalla(CodigoFalla codigo) {
        this.codigosCapturados.add(codigo);
    }

    public boolean tieneFallasCriticas() {
        return !this.codigosCapturados.isEmpty();
    }
}
```

2.1.6. Entidad Cotizacion

Presupuesto detallado que especifica costos de mano de obra, repuestos requeridos e inactividad antes de cualquier intervención.

Atributos:

- CotizacionId id (Value Object de Identidad)
- VehiculoId vehiculoId (Value Object de Identidad)

- EstadoCotizacion estado (BORRADOR, PENDIENTE_APROBACION, APROBADA, RECHAZADA)
- List<ItemCotizacion> items (Mano de obra y Repuestos requeridos) (Value Object ItemCotización)
- Monto totalEstimado (Value Object: Moneda y Valor)
- int diasInactividadEstimados
- DateTime fechaCreacion

Métodos:

- agregarItem(String descripcion, Monto precioUnitario, int cantidad)
- aprobar(AutorizacionCliente autorizacion) (Aplica Regla de Negocio 1: Requiere autorización explícita del cliente) Recibe el (Value Object de Datos: AutorizacionCliente) para validar la firma digital.

Código:

```
public class Cotizacion {
    private final CotizacionId id;
    private VehiculoId vehiculoId;
    private EstadoCotizacion estado;
    private List<ItemCotizacion> items; // Value Object de Datos
    private Monto totalEstimado;
    private int diasInactividadEstimados;
    private LocalDateTime fechaCreacion;

    public Cotizacion(CotizacionId id, VehiculoId vehiculoId) {
        this.id = id;
        this.vehiculoId = vehiculoId;
        this.estado = EstadoCotizacion.BORRADOR;
        this.items = new ArrayList<>();
        this.fechaCreacion = LocalDateTime.now();
    }

    public void agregarItem(ItemCotizacion item) {
        this.items.add(item);
    }

    public void aprobar(AutorizacionCliente autorizacion) {
        if (autorizacion == null) {
            throw new IllegalArgumentException("Se requiere autorización
explícita del cliente.");
        }
        this.estado = EstadoCotizacion.APROBADA;
    }
}
```

2.2. Contexto Acotado: Operaciones de Taller (Core Domain)

2.2.1. Entidad OrdenTrabajo

Instrucción formal que habilita y guía la intervención técnica del vehículo en el taller.

Atributos:

- OrdenTrabajoId id (Value Object de Identidad)
- CotizacionId cotizacionId (Value Object de Identidad)
- VehiculoId vehiculoId (Value Object de Identidad)
- TecnicoId tecnicoAsignadoId (Value Object de Identidad)
- BahiaTrabajoId bahiaId (Value Object de Identidad)

- EstadoOrdenTrabajo estado (EN_ESPERA, EN_PROCESO, BLOQUEADO_POR_REPUESTO, FINALIZADO)
- List<TareaReparacion> tareas
- DateTime fechaInicio
- DateTime fechaFinEstimada

Métodos:

- asignarBahia(BahiaTrabajoId bahiaId)
- bloquearPorFaltaRepuesto(String motivo) (Protección frente al Stock Fantasma)
- completarTarea(TareaReparacionId tareaId)
- marcarComoListaParaCalidad()

Código:

```
public class OrdenTrabajo {
   private final OrdenTrabajoId id;
   private CotizacionId cotizacionId;
   private VehiculoId vehiculoId;
   private TecnicoId tecnicoAsignadoId;
   private BahiaTrabajoId bahiaId;
   private EstadoOrdenTrabajo estado;
   private List<TareaReparacion> tareas;
   private LocalDateTime fechaInicio;
   private LocalDateTime fechaFinEstimada;

   public OrdenTrabajo(OrdenTrabajoId id, CotizacionId cotizacionId,
VehiculoId vehiculoId, LocalDateTime fechaFinEstimada) {
       this.id = id;
       this.cotizacionId = cotizacionId;
       this.vehiculoId = vehiculoId;
       this.fechaFinEstimada = fechaFinEstimada;
       this.estado = EstadoOrdenTrabajo.EN_ESPERA;
       this.tareas = new ArrayList<>();
   }

   public void asignarBahia(BahiaTrabajoId bahiaId) {
       this.bahiaId = bahiaId;
       this.estado = EstadoOrdenTrabajo.EN_PROCESO;
       this.fechaInicio = LocalDateTime.now();
   }

   public void bloquearPorFaltaRepuesto(String motivo) {
       this.estado = EstadoOrdenTrabajo.BLOQUEADO_POR_REPUESTO;
   }

   public void completarTarea(TareaReparacionId tareaId) {
       for (TareaReparacion tarea : tareas) {
           if (tarea.getId().equals(tareaId)) {
               break;
           }
       }
   }

   public void marcarComoListaParaCalidad() {
       this.estado = EstadoOrdenTrabajo.FINALIZADO;
   }
}
```

2.2.2. Entidad TareaReparacion

Unidad de trabajo mecánico específica dentro de la Orden de Trabajo.

Atributos:

- TareaReparacionId id (Value Object de Identidad)
- String descripcion
- EstadoTarea estado (PENDIENTE, EN_EJECUCION, COMPLETADA)
- List<FotoEvidencia> evidencias
- List<RegistroUsoRepuesto> repuestosUtilizados

Métodos:

- adjuntarEvidenciaFotografica(String urlFoto, String notaTecnica)
- registrarConsumoRepuesto(RepuestoId repuestoId, int cantidad)

Código:

```
public class TareaReparacion {
    private final TareaReparacionId id;
    private String descripcion;
    private EstadoTarea estado;
    private List<FotoEvidencia> evidencias;
    private List<RegistroUsoRepuesto> repuestosUtilizados;

    public TareaReparacion(TareaReparacionId id, String descripcion) {
        this.id = id;
        this.descripcion = descripcion;
        this.estado = EstadoTarea.PENDIENTE;
        this.evidencias = new ArrayList<>();
        this.repuestosUtilizados = new ArrayList<>();
    }

    public void adjuntarEvidenciaFotografica(FotoEvidencia evidencia) {
        this.evidencias.add(evidencia);
    }

    public void registrarConsumoRepuesto(RegistroUsoRepuesto uso) {
        this.repuestosUtilizados.add(uso);
    }
}
```

2.2.3. Entidad BahiaTrabajo

Espacio físico calibrado del taller asignado a un vehículo.

Atributos:

- BahiaTrabajoId id (Value Object de Identidad)
- String codigo (ej. "BAHIA-01")
- TipoBahia tipo (MECANICA_GENERAL, DIAGNOSTICO_ELECTRICO, ELEVADOR)
- EstadoBahia estado (LIBRE, OCUPADA, MANTENIMIENTO)

Métodos:

- ocuparConOrden(OrdenTrabajoId ordenId)
- liberar()

Código:

```
public class BahiaTrabajo {
    private final BahiaTrabajoId id;
    private String codigo;
    private TipoBahia tipo;
```

```
    private EstadoBahia estado;

    public BahiaTrabajo(BahiaTrabajoId id, String codigo, TipoBahia
tipo) {
        this.id = id;
        this.codigo = codigo;
        this.tipo = tipo;
        this.estado = EstadoBahia.LIBRE;
    }

    public void ocuparConOrden(OrdenTrabajoId ordenId) {
        this.estado = EstadoBahia.OCUPADA;
    }

    public void liberar() {
        this.estado = EstadoBahia.LIBRE;
    }
}
```

2.3. Contexto Acotado: Control de Calidad y Entrega (Core Domain)

2.3.1. Entidad PruebaCalidad

Evaluación estricta realizada por un supervisor independiente antes de autorizar la salida del vehículo.

Atributos:

- PruebaCalidadId id (Value Object de Identidad)
- OrdenTrabajoId ordenTrabajoId (Value Object de Identidad)
- SupervisorCalidadId supervisorId (Value Object de Identidad)
- boolean apruebaPruebaEstatica (Luces, fugas, torque)
- boolean apruebaPruebaRuta
- EstadoPruebaCalidad estado (EN_REVISION, APROBADO, RECHAZADO_A_RETRABAJO)
- String observaciones

Métodos:

- registrarResultadoEstatico(boolean exitoso, String detalles)
- aprobarControlCalidad()
- rechazarYEnviarARetrabajo(String motivo)

Código:

```
public class PruebaCalidad {
    private final PruebaCalidadId id;
    private OrdenTrabajoId ordenTrabajoId;
    private SupervisorCalidadId supervisorId;
    private boolean apruebaPruebaEstatica;
    private boolean apruebaPruebaRuta;
    private EstadoPruebaCalidad estado;
    private String observaciones;

    public PruebaCalidad(PruebaCalidadId id, OrdenTrabajoId
ordenTrabajoId, SupervisorCalidadId supervisorId) {
        this.id = id;
        this.ordenTrabajoId = ordenTrabajoId;
        this.supervisorId = supervisorId;
        this.apruebaPruebaEstatica = false;
        this.apruebaPruebaRuta = false;
```

```
        this.estado = EstadoPruebaCalidad.EN_REVISION;
        this.observaciones = "";
    }

    public void registrarResultadoEstatico(boolean exitoso, String
detalles) {
        this.apruebaPruebaEstatica = exitoso;
        this.observaciones += " [Estática: " + detalles + "]";
    }

    public void aprobarControlCalidad() {
        this.apruebaPruebaRuta = true;
        this.estado = EstadoPruebaCalidad.APROBADO;
    }

    public void rechazarYEnviarARetrabajo(String motivo) {
        this.estado = EstadoPruebaCalidad.RECHAZADO_A_RETRABAJO;
        this.observaciones += " [Motivo Rechazo: " + motivo + "]";
    }
}
```

2.3.2. Entidad InformeEntrega

Documento de cierre del servicio que detalla el trabajo realizado, piezas sustituidas y condiciones de garantía.

Atributos:

- InformeEntregaId id (Value Object de Identidad)
- OrdenTrabajoId ordenTrabajoId (Value Object de Identidad)
- DateTime fechaEntrega
- CondicionGarantia garantiaOtorgada (Value Object CondicionGarantia)
- RegistroMantenimientoSugerido proximoMantenimiento
- boolean vehiculoLavado

Método:

- programarProximoMantenimiento(int kilometrajeSugerido, DateTime fechaEstimada)

Código:

```
public class InformeEntrega {
    private final InformeEntregaId id;
    private OrdenTrabajoId ordenTrabajoId;
    private LocalDateTime fechaEntrega;
    private CondicionGarantia garantiaOtorgada;
    private RegistroMantenimientoSugerido proximoMantenimiento;
    private boolean vehiculoLavado;

    public InformeEntrega(InformeEntregaId id, OrdenTrabajoId
ordenTrabajoId, CondicionGarantia garantiaOtorgada) {
        this.id = id;
        this.ordenTrabajoId = ordenTrabajoId;
        this.garantiaOtorgada = garantiaOtorgada;
        this.fechaEntrega = LocalDateTime.now();
        this.vehiculoLavado = true;
    }

    public void programarProximoMantenimiento(int kilometrajeSugerido,
LocalDateTime fechaEstimada) {
        this.proximoMantenimiento = new
RegistroMantenimientoSugerido(kilometrajeSugerido, fechaEstimada);
```

```
    }
}
```

2.4. Contexto Acotado: Inventario y Almacén (Supporting Subdomain)

2.4.1. Entidad Repuesto

Componente físico o insumo catalogado en el inventario logístico del taller.

Atributos:

- RepuestoId id (Value Object de Identidad)
- String codigoSku
- String nombre
- int stockFisicoReal
- int stockReservado
- UmbralReposicion umbral (Value Object UmbralReposicion)
- Monto precioUnitario (Value Object: Moneda y Valor)

Métodos:

- reservarStock(int cantidad): Disminuye disponible y aumenta reservado.
- despacharStock(int cantidad): Reduce stockFisicoReal y stockReservado.
- evaluarUmbralReposicion(): boolean: (Aplica Regla de Negocio 4: Si (stockFisicoReal - reservado) < umbral, dispara alerta/pedido).

Código:

```
public class Repuesto {
    private final RepuestoId id;
    private String codigoSku;
    private String nombre;
    private int stockFisicoReal;
    private int stockReservado;
    private UmbralReposicion umbral;
    private Monto precioUnitario;

    public Repuesto(RepuestoId id, String codigoSku, String nombre,
UmbralReposicion umbral, Monto precioUnitario) {
        this.id = id;
        this.codigoSku = codigoSku;
        this.nombre = nombre;
        this.umbral = umbral;
        this.precioUnitario = precioUnitario;
        this.stockFisicoReal = 0;
        this.stockReservado = 0;
    }

    public void reservarStock(int cantidad) {
        this.stockReservado += cantidad;
    }

    public void despacharStock(int cantidad) {
        if (cantidad > this.stockReservado || cantidad >
this.stockFisicoReal) {
            throw new IllegalArgumentException("No se puede despachar
más stock del físicamente disponible o reservado.");
        }
        this.stockFisicoReal -= cantidad;
        this.stockReservado -= cantidad;
```

```
    }

    public boolean evaluarUmbralReposicion() {
        return (this.stockFisicoReal - this.stockReservado) <
this.umbral.getCantidadMinima();
    }
}
```

2.4.2. Entidad ReservaRepuesto

Apartado preventivo de stock asociado a una cotización aprobada para evitar Stock Fantasma.

Atributos:

- ReservaId id (Value Object de Identidad)
- CotizacionId cotizacionId (Value Object de Identidad)
- RepuestoId repuestoId (Value Object de Identidad)
- int cantidad
- EstadoReserva estado (ACTIVA, CONSUMIDA, CANCELADA)
- DateTime fechaExpiracion

Métodos:

- marcarComoConsumida()
- cancelarReserva()

Código:

```
public class ReservaRepuesto {
    private final ReservaId id;
    private CotizacionId cotizacionId;
    private RepuestoId repuestoId;
    private int cantidad;
    private EstadoReserva estado;
    private LocalDateTime fechaExpiracion;

    public ReservaRepuesto(ReservaId id, CotizacionId cotizacionId,
RepuestoId repuestoId, int cantidad, LocalDateTime fechaExpiracion) {
        this.id = id;
        this.cotizacionId = cotizacionId;
        this.repuestoId = repuestoId;
        this.cantidad = cantidad;
        this.fechaExpiracion = fechaExpiracion;
        this.estado = EstadoReserva.ACTIVA;
    }

    public void marcarComoConsumida() {
        this.estado = EstadoReserva.CONSUMIDA;
    }

    public void cancelarReserva() {
        this.estado = EstadoReserva.CANCELADA;
    }
}
```

2.5. Contexto Acotado: Agendamiento (Supporting Subdomain)

2.5.1. Entidad Cita

Reserva previa de turno de atención en recepción.

Atributos:

- CitaId id (Value Object de Identidad)
- ClienteId clienteId (Value Object de Identidad)
- VehiculoId vehiculoId (Value Object de Identidad)
- DateTime fechaProgramada
- SlotHorario bloqueHorario (Value Object SlotHorario)
- EstadoCita estado (PROGRAMADA, CONFIRMADA, ATENDIDA, CANCELADA)

Métodos:

- confirmarCita()
- marcarComoAtendida()

Código:

```
public class Cita {
    private final CitaId id;
    private ClienteId clienteId;
    private VehiculoId vehiculoId;
    private LocalDateTime fechaProgramada;
    private SlotHorario bloqueHorario;
    private EstadoCita estado;

    public Cita(CitaId id, ClienteId clienteId, VehiculoId vehiculoId,
SlotHorario bloque) {
        this.id = id;
        this.clienteId = clienteId;
        this.vehiculoId = vehiculoId;
        this.bloqueHorario = bloque;
        this.estado = EstadoCita.PROGRAMADA;
    }

    public void confirmarCita() {
        this.estado = EstadoCita.CONFIRMADA;
    }

    public void marcarComoAtendida() {
        this.estado = EstadoCita.ATENDIDA;
    }
}
```

2.6. Contexto Acotado: Gestión de Personal (Supporting Subdomain)

2.6.1. Entidad Tecnico

Personal operativo del taller.

Atributos:

- TecnicoId id (Value Object de Identidad)
- String dni
- String nombreCompleto
- List<Especialidad> especialidades (ej. MECANICA_LIGERA, DIAGNOSTICO_OBD2, FRENOS)
- EstadoDisponibilidad estado (DISPONIBLE, OCUPADO, EN_DESCANSO)
- TurnoHorario turnoLaboral (Value Object TurnoHorario)

Métodos:

- asignarAOrden()
- liberarTecnico()

Código:

```
public class Tecnico {
   private final TecnicoId id;
   private String dni;
   private String nombreCompleto;
   private List<Especialidad> especialidades;
   private EstadoDisponibilidad estado;
   private TurnoHorario turnoLaboral;

   public Tecnico(TecnicoId id, String dni, String nombreCompleto,
List<Especialidad> especialidades, TurnoHorario turnoLaboral) {
       this.id = id;
       this.dni = dni;
       this.nombreCompleto = nombreCompleto;
       this.especialidades = especialidades;
       this.turnoLaboral = turnoLaboral;
       this.estado = EstadoDisponibilidad.DISPONIBLE;
   }

   public void asignarAOrden() {
       this.estado = EstadoDisponibilidad.OCUPADO;
   }

   public void liberarTecnico() {
       this.estado = EstadoDisponibilidad.DISPONIBLE;
   }
}
```

2.7. Contexto Acotado: Facturación y Finanzas (Generic Subdomain)

2.7.1. Entidad ComprobantePago

Documento tributario electrónico emitido bajo las regulaciones nacionales de SUNAT.

Atributos:

- ComprobanteId id (Value Object de Identidad)
- InformeEntregaId informeEntregaId (Value Object de Identidad)
- TipoComprobante tipo (BOLETA, FACTURA)
- String serieNumero (ej. "F001-00004523")
- Monto montoSubtotal
- Monto montoIgv
- Monto montoTotal
- EstadoFactura estado (EMITIDO, ANULADO, ENVIADO_SUNAT)
- String xmlFirmadoUrl

Métodos:

- generarXMLSunat()
- anularComprobante(String motivo)

Código:

```
public class ComprobantePago {
   private final ComprobanteId id;
   private InformeEntregaId informeEntregaId;
   private TipoComprobante tipo;
   private String serieNumero;
   private Monto montoSubtotal;
```

```
    private Monto montoIgv;
    private Monto montoTotal;
    private EstadoFactura estado;
    private String xmlFirmadoUrl;

    public ComprobantePago(ComprobanteId id, InformeEntregaId
informeEntregaId, TipoComprobante tipo,
                           String serieNumero, Monto montoSubtotal,
Monto montoIgv, Monto montoTotal) {
        this.id = id;
        this.informeEntregaId = informeEntregaId;
        this.tipo = tipo;
        this.serieNumero = serieNumero;
        this.montoSubtotal = montoSubtotal;
        this.montoIgv = montoIgv;
        this.montoTotal = montoTotal;
        this.estado = EstadoFactura.EMITIDO;
        this.xmlFirmadoUrl = null; // Aún no firmado por SUNAT al
instanciar
    }

    public void generarXMLSunat() {
        // Construir el XML y enviarlo a SUNAT
        this.xmlFirmadoUrl = "https://facturacion.sunat.gob.pe/xml/" +
this.serieNumero + ".xml";
        this.estado = EstadoFactura.ENVIADO_SUNAT;
    }

    public void anularComprobante(String motivo) {
        this.estado = EstadoFactura.ANULADO;
    }
}
```

3. Descripción de Objetos de valor

3.1. Contexto Acotado: Recepción y Diagnóstico

3.1.1. Value Object Placa

La placa vehicular es un dato estructurado con un formato de negocio propio (tres letras y tres números), pero no se utiliza como identificador técnico del Vehiculo porque puede reasignarse a lo largo del tiempo (robo, deterioro, transferencia). Se modela como Value Object porque dos vehículos con la misma cadena de placa representan el mismo valor de placa, y su igualdad se determina por el contenido, no por una identidad propia.

```
public class Placa {
    private static final String PATRON = "([A-Z]{3}-\\d{3})|([A-Z]\\d-\\d{4})|([A-Z]{2}-\\d{4})";
    private final String valor;

    public Placa(String valor) {
        if (valor == null || !valor.matches(PATRON))
            throw new IllegalArgumentException("Formato de placa no reconocido");
        this.valor = valor;
    }
}
```

3.1.2. Value Object Kilometraje

El kilometraje describe una magnitud del vehículo, no tiene identidad propia. Encapsularlo dentro de un Value Object garantiza que sea imposible construir o actualizar un kilometraje que decrezca respecto al valor anterior.

```
public class Kilometraje {
    private final int valor;

    public Kilometraje(int valor) {
        if (valor < 0) throw new IllegalArgumentException("Kilometraje no puede ser negativo");
        this.valor = valor;
    }

    public Kilometraje actualizar(int nuevoValor) {
        if (nuevoValor < this.valor)
            throw new IllegalArgumentException("El kilometraje no puede decrecer");
        return new Kilometraje(nuevoValor);
    }
}
```

3.1.3. Value Object CodigoFalla

El código de falla es un dato que el escáner captura y que solo tiene significado dentro del diagnóstico en el que aparece. No tiene un ciclo de vida propio ni continuidad más allá de ese registro. Si el mismo código aparece en dos diagnósticos distintos, ambos representan la misma falla técnica, no hace falta distinguirlos con una identidad separada.

```
public class CodigoFalla {
    private final String codigo;
    private final String descripcion;

    public CodigoFalla(String codigo, String descripcion) {
        if (codigo == null || !codigo.matches("[PBCU]\\d{4}"))
            throw new IllegalArgumentException("Formato de código OBD-II inválido");
        this.codigo = codigo;
        this.descripcion = descripcion;
    }
}
```

3.1.4. Value Object ItemCotizacion

Representa una línea de mano de obra o repuesto dentro de una cotización. No tiene ciclo de vida propio fuera de la Cotización que lo contiene, no se consulta, edita ni elimina un ítem por su propia identidad. Dos ítems con la misma descripción, precio y cantidad son intercambiables.

```
public class ItemCotizacion {
    private final String descripcion;
    private final Monto precioUnitario;
    private final int cantidad;

    public ItemCotizacion(String descripcion, Monto precioUnitario, int cantidad) {
        if (cantidad <= 0)
            throw new IllegalArgumentException("La cantidad debe ser mayor a cero");
        this.descripcion = descripcion;
        this.precioUnitario = precioUnitario;
        this.cantidad = cantidad;
    }
}
```

3.1.5. Value Object AutorizacionCliente

Describe las condiciones bajo las cuales el cliente autorizó la intervención (Regla de Negocio 1), el método utilizado, el momento y la evidencia asociada. No es una entidad con estado propio ni continuidad en el tiempo, sino un dato inmutable que se adjunta al momento de aprobar la Cotizacion.

```
public class AutorizacionCliente {
    private final String metodoAutorizacion;
    private final DateTime fechaHora;
    private final String evidenciaUrl;

    public AutorizacionCliente(String metodoAutorizacion, DateTime fechaHora, String evidenciaUrl) {
        if (evidenciaUrl == null || evidenciaUrl.isBlank())
            throw new IllegalArgumentException("Se requiere evidencia de la autorización");
        this.metodoAutorizacion = metodoAutorizacion;
        this.fechaHora = fechaHora;
        this.evidenciaUrl = evidenciaUrl;
    }
}
```

3.1.6. Value Object Monto

El dinero no tiene identidad propia. Dos montos de S/150.00 son completamente intercambiables entre cotizaciones, repuestos o comprobantes de pago, por lo que no existe ninguna razón de negocio para diferenciarlos con un identificador. Se origina en este contexto y se reutiliza de forma transversal en Inventario y Almacén y en Facturación y Finanzas.

```java
public class Monto {
    private final BigDecimal valor;
    private final String moneda;

    public Monto(BigDecimal valor, String moneda) {
        if (valor.compareTo(BigDecimal.ZERO) < 0)
            throw new IllegalArgumentException("El monto no puede ser negativo");
        this.valor = valor;
        this.moneda = moneda;
    }
}
```

3.1.7. Value Object ContactoCliente

Agrupa los medios de contacto del cliente (teléfono y correo electrónico) como un dato compuesto.

```java
public class ContactoCliente {
    private final String telefono;
    private final String correo;

    public ContactoCliente(String telefono, String correo) {
        if (telefono == null || !telefono.matches("\\d{9}"))
            throw new IllegalArgumentException("Teléfono inválido, se esperan 9 dígitos");
        if (correo == null || !correo.matches("^[\\w.+-]+@[\\w-]+\\.[a-zA-Z]{2,}$"))
            throw new IllegalArgumentException("Correo electrónico inválido");
        this.telefono = telefono;
        this.correo = correo;
    }
}
```

3.1.8. Value Objects de Identidad

A diferencia de los Value Objects de datos, estos no representan un atributo de negocio, sino el identificador de una Entidad. Se modelan como Value Object porque son inmutables y su igualdad se determina por el valor que contienen, no por una identidad propia.

El tipo interno que envuelven estos Value Objects es un UUID, un identificador de 128 bits que se genera automáticamente y prácticamente nunca se repite, sin depender de que la base de datos lleve el conteo de los Ids ya usados [1]. Esto permite que cada Entidad reciba su identificador desde que se crea, incluso antes de guardarse, de modo que una Entidad pueda referenciar a otra solo con su Id

3.1.8.1. VehiculoId

```java
public class VehiculoId {
    private final UUID valor;

    public VehiculoId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof VehiculoId)) return false;
        return valor.equals(((VehiculoId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.1.8.2. ClienteId

```java
public class ClienteId {
    private final UUID valor;

    public ClienteId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof ClienteId)) return false;
        return valor.equals(((ClienteId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.1.8.3. RecepcionId

```java
public class RecepcionId {
    private final UUID valor;

    public RecepcionId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof RecepcionId)) return false;
        return valor.equals(((RecepcionId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.1.8.4. InspeccionFisicaId

```java
public class InspeccionFisicaId {
    private final UUID valor;

    public InspeccionFisicaId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof InspeccionFisicaId)) return false;
        return valor.equals(((InspeccionFisicaId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.1.8.5. DiagnosticoId

```java
public class DiagnosticoId {
    private final UUID valor;

    public DiagnosticoId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof DiagnosticoId)) return false;
        return valor.equals(((DiagnosticoId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.1.8.6. CotizacionId

```java
public class CotizacionId {
    private final UUID valor;

    public CotizacionId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof CotizacionId)) return false;
        return valor.equals(((CotizacionId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.2. Contexto Acotado: Operaciones de Taller

3.2.1. Value Object FotoEvidencia

Documenta el respaldo fotográfico del avance de una reparación, la URL de la imagen y la nota técnica asociada.

```
public class FotoEvidencia {
    private final String urlFoto;
    private final String notaTecnica;

    public FotoEvidencia(String urlFoto, String notaTecnica) {
        if (urlFoto == null || urlFoto.isBlank())
            throw new IllegalArgumentException("La URL de la evidencia es obligatoria");
        this.urlFoto = urlFoto;
        this.notaTecnica = notaTecnica;
    }
}
```

3.2.2. Value Object RegistroUsoRepuesto

Documenta el consumo de un repuesto dentro de una tarea de reparación, qué pieza, en qué cantidad y cuándo se registró.

```
public class RegistroUsoRepuesto {
    private final RepuestoId repuestoId;
    private final int cantidad;
    private final DateTime fechaRegistro;

    public RegistroUsoRepuesto(RepuestoId repuestoId, int cantidad, DateTime fechaRegistro) {
        if (cantidad <= 0)
            throw new IllegalArgumentException("Cantidad debe ser positiva");
        this.repuestoId = repuestoId;
        this.cantidad = cantidad;
        this.fechaRegistro = fechaRegistro;
    }
}
```

3.2.3. Value Objects de Identidad

3.2.3.1. OrdenTrabajoId

```java
public class OrdenTrabajoId {
    private final UUID valor;

    public OrdenTrabajoId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof OrdenTrabajoId)) return false;
        return valor.equals(((OrdenTrabajoId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.2.3.2. TecnicoId

```java
public class TecnicoId {
    private final UUID valor;

    public TecnicoId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof TecnicoId)) return false;
        return valor.equals(((TecnicoId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.2.3.3. BahiaTrabajoId

```java
public class BahiaTrabajoId {
    private final UUID valor;

    public BahiaTrabajoId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof BahiaTrabajoId)) return false;
        return valor.equals(((BahiaTrabajoId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.2.3.4. TareaReparacionId

```java
public class TareaReparacionId {
    private final UUID valor;

    public TareaReparacionId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof TareaReparacionId)) return false;
        return valor.equals(((TareaReparacionId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.3. Contexto Acotado: Control de Calidad y Entrega

3.3.1. Value Object CondicionGarantia

Corresponde a la Regla de Negocio 3 (garantía total de 3 meses o 5,000 km). No tiene identidad propia porque dos garantías con la misma duración y kilometraje son el mismo valor, no hay nada que distinguir entre una y otra.

```
public class CondicionGarantia {
    private final int duracionMeses;
    private final int kilometrosCobertura;

    public CondicionGarantia(int duracionMeses, int kilometrosCobertura) {
        if (duracionMeses <= 0 || kilometrosCobertura <= 0)
            throw new IllegalArgumentException("Duración y cobertura deben ser positivas");
        this.duracionMeses = duracionMeses;
        this.kilometrosCobertura = kilometrosCobertura;
    }
}
```

3.3.2. Value Object RegistroMantenimientoSugerido

Describe la recomendación de próximo mantenimiento (kilometraje y fecha estimada) que se adjunta al InformeEntrega. No representa una cita ni una entidad rastreable de forma independiente, sino un dato descriptivo generado en el momento de la entrega.

```
public class RegistroMantenimientoSugerido {
    private final int kilometrajeSugerido;
    private final DateTime fechaEstimada;

    public RegistroMantenimientoSugerido(int kilometrajeSugerido, DateTime fechaEstimada) {
        if (kilometrajeSugerido <= 0)
            throw new IllegalArgumentException("Kilometraje sugerido inválido");
        this.kilometrajeSugerido = kilometrajeSugerido;
        this.fechaEstimada = fechaEstimada;
    }
}
```

3.3.3. Value Object CheckSeguridad

Representa el resultado de una verificación puntual dentro de la prueba de calidad (luces, fugas,etc).

```java
public class CheckSeguridad {
    private final String nombreCheck;
    private final boolean aprobado;
    private final String observacion;

    public CheckSeguridad(String nombreCheck, boolean aprobado, String observacion) {
        if (nombreCheck == null || nombreCheck.isBlank())
            throw new IllegalArgumentException("El check debe tener un nombre");
        this.nombreCheck = nombreCheck;
        this.aprobado = aprobado;
        this.observacion = observacion;
    }

    public boolean isAprobado() {
        return aprobado;
    }
}
```

3.3.4. Value Objects de Identidad

3.3.4.1. PruebaCalidadId

```java
public class PruebaCalidadId {
    private final UUID valor;

    public PruebaCalidadId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof PruebaCalidadId)) return false;
        return valor.equals(((PruebaCalidadId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.3.4.2. SupervisorCalidadId

```java
public class SupervisorCalidadId {
    private final UUID valor;

    public SupervisorCalidadId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof SupervisorCalidadId)) return false;
        return valor.equals(((SupervisorCalidadId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.3.4.3. InformeEntregaId

```java
public class InformeEntregaId {
    private final UUID valor;

    public InformeEntregaId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof InformeEntregaId)) return false;
        return valor.equals(((InformeEntregaId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.4. Contexto Acotado: Inventario y Almacén

3.4.1. Value Object UmbralReposicion

El umbral describe una condición de reabastecimiento, no una pieza física que haya que identificar. Dos repuestos distintos podrían compartir exactamente el mismo umbral (misma cantidad mínima, mismos días de cobertura) sin que eso genere ningún conflicto.

```
public class UmbralReposicion {
    private final int cantidadMinima;
    private final int diasCobertura;

    public UmbralReposicion(int cantidadMinima, int diasCobertura) {
        if (cantidadMinima < 0 || diasCobertura <= 0)
            throw new IllegalArgumentException("Umbral inválido");
        this.cantidadMinima = cantidadMinima;
        this.diasCobertura = diasCobertura;
    }
}
```

3.4.2. Value Object Monto

(ver 3.1.6)

3.4.3. Value Objects de Identidad

3.4.3.1. RepuestoId

```
public class RepuestoId {
    private final UUID valor;

    public RepuestoId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof RepuestoId)) return false;
        return valor.equals(((RepuestoId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.4.3.2. ReservaId

```java
public class ReservaId {
    private final UUID valor;

    public ReservaId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof ReservaId)) return false;
        return valor.equals(((ReservaId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.5. Contexto Acotado: Agendamiento

3.5.1. Value Object SlotHorario

Un bloque horario se define completamente por su hora de inicio y de fin. No tiene ninguna propiedad adicional que lo haga único más allá de esos dos valores. Que dos citas distintas usen el mismo rango horario no genera conflicto alguno, porque cada Cita ya tiene su propia identidad, el slot solo describe el horario que ocupa.

```java
public class SlotHorario {
    private final LocalTime horaInicio;
    private final LocalTime horaFin;

    public SlotHorario(LocalTime horaInicio, LocalTime horaFin) {
        if (!horaFin.isAfter(horaInicio))
            throw new IllegalArgumentException("horaFin debe ser posterior a horaInicio");
        this.horaInicio = horaInicio;
        this.horaFin = horaFin;
    }
}
```

3.5.2. Value Objects de Identidad

3.5.2.1. CitaId

```java
public class CitaId {
    private final UUID valor;

    public CitaId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof CitaId)) return false;
        return valor.equals(((CitaId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

3.6. Contexto Acotado: Gestión de Personal

3.6.1. Value Object TurnoHorario

Define el bloque de jornada laboral de un técnico (día y rango horario).

```java
public class TurnoHorario {
    private final DayOfWeek dia;
    private final LocalTime horaInicio;
    private final LocalTime horaFin;

    public TurnoHorario(DayOfWeek dia, LocalTime horaInicio, LocalTime horaFin) {
        if (!horaFin.isAfter(horaInicio))
            throw new IllegalArgumentException("Turno inválido");
        this.dia = dia;
        this.horaInicio = horaInicio;
        this.horaFin = horaFin;
    }
}
```

3.6.2. Value Objects de Identidad

3.6.2.1. TecnicoId

```java
public class TecnicoId {
  private final UUID valor;

  public TecnicoId(UUID valor) {
    if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
    this.valor = valor;
  }

  @Override
  public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof TecnicoId)) return false;
    return valor.equals(((TecnicoId) o).valor);
  }

  @Override
  public int hashCode() { return valor.hashCode(); }
}
```

3.7. Contexto Acotado: Facturación y Finanzas

3.7.1. Value Object Monto

(ver 3.1.6)

3.7.2. Value Objects de Identidad

3.7.2.1. ComprobanteId

```java
public class ComprobanteId {
  private final UUID valor;

  public ComprobanteId(UUID valor) {
    if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
    this.valor = valor;
  }

  @Override
  public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof ComprobanteId)) return false;
    return valor.equals(((ComprobanteId) o).valor);
  }

  @Override
  public int hashCode() { return valor.hashCode(); }
}
```

3.7.2.2. InformeEntregaId

```java
public class InformeEntregaId {
    private final UUID valor;

    public InformeEntregaId(UUID valor) {
        if (valor == null) throw new IllegalArgumentException("Id no puede ser nulo");
        this.valor = valor;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof InformeEntregaId)) return false;
        return valor.equals(((InformeEntregaId) o).valor);
    }

    @Override
    public int hashCode() { return valor.hashCode(); }
}
```

4. Descripción de Agregados

4.1. CONTEXTO ACOTADO: RECEPCIÓN Y DIAGNÓSTICO

4.1.1. AGREGADO 1: Cliente

Entidad Raíz

Cliente

Propósito

Representa a la persona natural o jurídica propietaria de los vehículos, con el cual se establecen las interacciones, cotizaciones y responsabilidades legales.

Invariantes / Reglas de Negocio Protegidas

Los datos de contacto no pueden ser nulos.

El DNI/RUC no puede ser vacío.

El nombre no puede ser vacío.

Entidades Internas

Nombre	Rol
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
ClienteId	Cliente	Sí
ContactoCliente	Cliente	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	ClienteId	Sí	Identificador único del cliente.
dniRuc	String	Sí	Documento de identidad o RUC.
nombreCompletoRazonSocial	String	Sí	Nombre o razón social del cliente.
contacto	ContactoCliente	Sí	Datos de contacto (teléfono, correo).
direccion	String	No	Dirección física del cliente.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Ninguna directa	-	-

Eventos de Dominio Publicados

Evento	Acción que lo dispara
Ninguno definido	N/A

Servicios de Dominio Relacionados

Ninguno.

Código

```java
/**
 * 1. AGREGADO RAÍZ
 * Representa al cliente del taller mecánico y gestiona su información base.
 */
public class Cliente {
    // Atributos de identificación y datos generales
    private ClienteId id;
    private String dniRuc;
    private String nombreCompletoRazonSocial;
    private String direccion;

    // Objeto de valor encapsulado
    private ContactoCliente contacto;

    // --- Comportamientos del Agregado ---

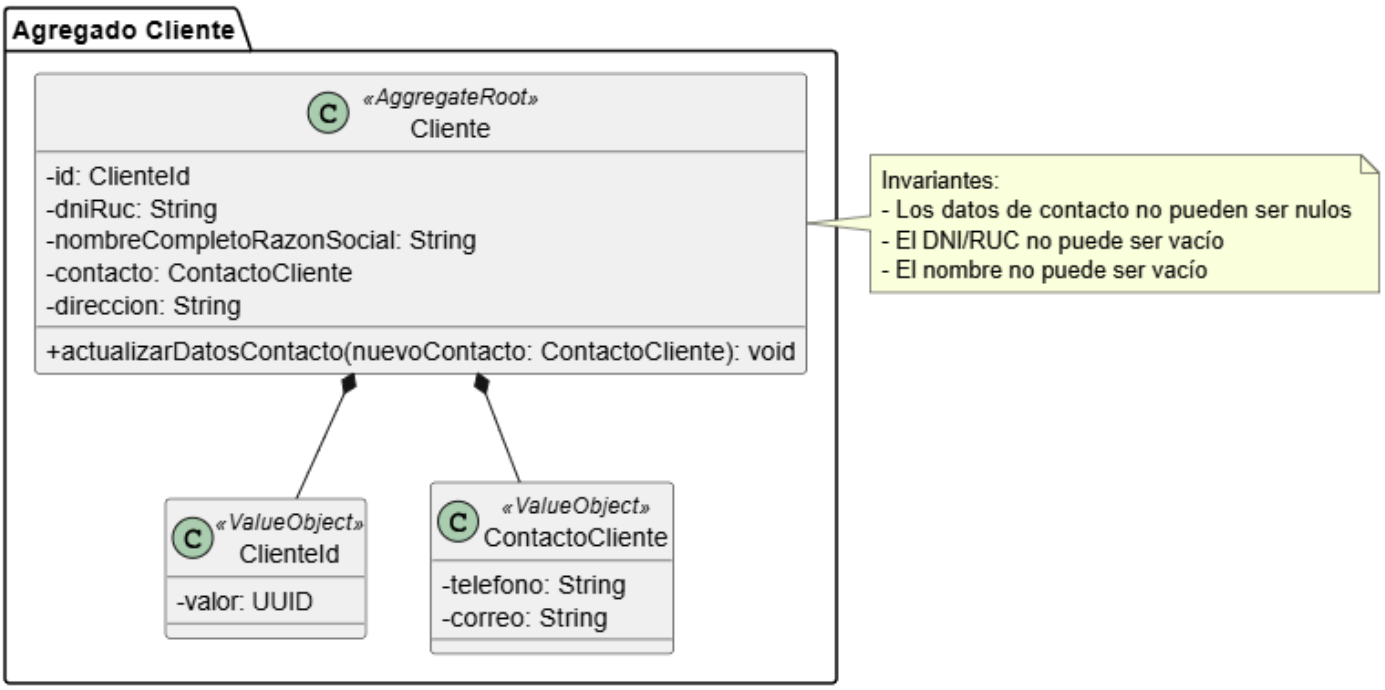
    // Permite la actualización de los medios de comunicación reemplazando el objeto de valor
    public void actualizarDatosContacto(ContactoCliente nuevoContacto) { /* ... */ }
}


/**
 * 2. OBJETOS DE VALOR (Value Objects)
 * Tipos de datos inmutables y reglas estructurales del dominio para la información del cliente.
 */

// Identificador fuertemente tipado (encapsula UUID)
record ClienteId(UUID valor) {}

// Objeto de valor que agrupa y valida los canales de comunicación del cliente
record ContactoCliente(String telefono, String correo) {
    /*
     * Reglas de validación estrictas (Constructor compacto):
     * - El 'telefono' debe coincidir con el patrón regex ^\d{9}$ (Exactamente 9 dígitos numéricos).
     * - El 'correo' debe tener un formato de email válido según el patrón regex ^[A-Za-z0-9+_.-]+@(.+)$.
     */
}
```

Diagrama



4.1.1. AGREGADO 2: RecepcionVehicular

Entidad Raíz

RecepcionVehicular

Propósito

Coordinar el ingreso del vehículo, capturando el estado físico inicial (inspección) y los posibles problemas electrónicos (diagnóstico OBD2), definiendo si el vehículo puede continuar a la cotización.

Invariantes / Reglas de Negocio Protegidas

No se puede completar la recepción sin una inspección física registrada.
No se puede completar la recepción sin diagnóstico OBD2 registrado.
La inspección física debe tener firma del cliente.

Entidades Internas

Nombre	Rol
InspeccionFisica	Registra daños, pertenencias y nivel de combustible al ingresar.
DiagnosticoOBD2	Registra los códigos de falla electrónicos encontrados por el escáner.

Objetos de Valor Utilizados

VO	Usado por	Compartido
RecepcionId	RecepcionVehicular	Sí
VehiculoId	RecepcionVehicular, InspeccionFisica, DiagnosticoOBD2	Sí
ClienteId	RecepcionVehicular	Sí
EstadoRecepcion	RecepcionVehicular	No
InspeccionFisicaId	InspeccionFisica	Sí
DiagnosticoId	DiagnosticoOBD2	Sí
CodigoFalla	DiagnosticoOBD2	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	RecepcionId	Sí	Identificador único de la recepción.
vehiculoId	VehiculoId	Sí	Vehículo recepcionado.
clienteId	ClienteId	Sí	Cliente que entrega el vehículo.

Atributo	Tipo	Obligatorio	Descripción
fechaIngreso	LocalDateTime	Sí	Fecha y hora de ingreso.
estado	EstadoRecepcion	Sí	Estado del proceso de recepción.
inspeccion	InspeccionFisica	No	Inspección física asociada.
diagnostico	DiagnosticoOBD2	No	Diagnóstico OBD2 asociado.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Vehiculo	VehiculoId	N:1
Cliente	ClienteId	N:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
InspeccionFisicaRegistrada	Al guardar la inspección física.
DiagnosticoOBD2Registrado	Al guardar el diagnóstico OBD2.
RecepcionVehicularCompletada	Al llamar a completarRecepcion().

Servicios de Dominio Relacionados

Ninguno directo.

Código

```java
/**
 * 1. AGREGADO RAÍZ
 * Coordina el proceso y asegura la consistencia de sus entidades internas.
 */
public class RecepcionVehicular {
    // Atributos de identificación y estado
    private RecepcionId id;
    private VehiculoId vehiculoId;
    private ClienteId clienteId;
    private LocalDateTime fechaIngreso;
    private EstadoRecepcion estado;

    // Entidades hijas
    private InspeccionFisica inspeccionFisica;
    private DiagnosticoOBD2 diagnosticoOBD2;

    // --- Comportamientos del Agregado ---
    public void agregarInspeccionFisica(InspeccionFisica inspeccion) { /*
... */ }

    public void agregarDiagnosticoOBD2(DiagnosticoOBD2 diagnostico) { /*
... */ }

    public void iniciarInspeccion() { /* Cambia estado a EN_INSPECCION */ }

    /*
     * Regla de negocio estricta:
     * Lanza excepción si no hay inspección, falta la firma del cliente,
     * o no hay diagnóstico registrado antes de pasar a COMPLETADO.
     */
    public void completarRecepcion() throws IllegalStateException { /* ...
*/ }
}


/**
 * 2. ENTIDADES INTERNAS
 * Manejan detalles específicos dentro del ciclo de vida de la Recepción.
 */
class InspeccionFisica {
    private InspeccionFisicaId id;
    private VehiculoId vehiculoId;
    private LocalDateTime fechaRegistro;
    private List<String> danosCarroceria;
    private boolean nivelCombustible;
```

```
    private List<String> pertenenciasDejadas;
    private String firmaClienteUrl;

    // Añade daño a la lista si el texto no es nulo/vacío
    public void registrarObservacion(String detalle) { /* ... */ }

    // Valida que la URL de la firma no esté vacía
    public void asociarFirmaCliente(String firmaUrl) throws
IllegalArgumentException { /* ... */ }
}

class DiagnosticoOBD2 {
    private DiagnosticoId id;
    private VehiculoId vehiculoId;
    private LocalDateTime fechaEscaneo;
    private List<CodigoFalla> codigosCapturados;
    private String notasTecnico;

    public void agregarCodigoFalla(CodigoFalla codigo) { /* ... */ }

    // Retorna true si la lista de codigosCapturados no está vacía
    public boolean tieneFallasCriticas() { /* ... */ return false; }
}



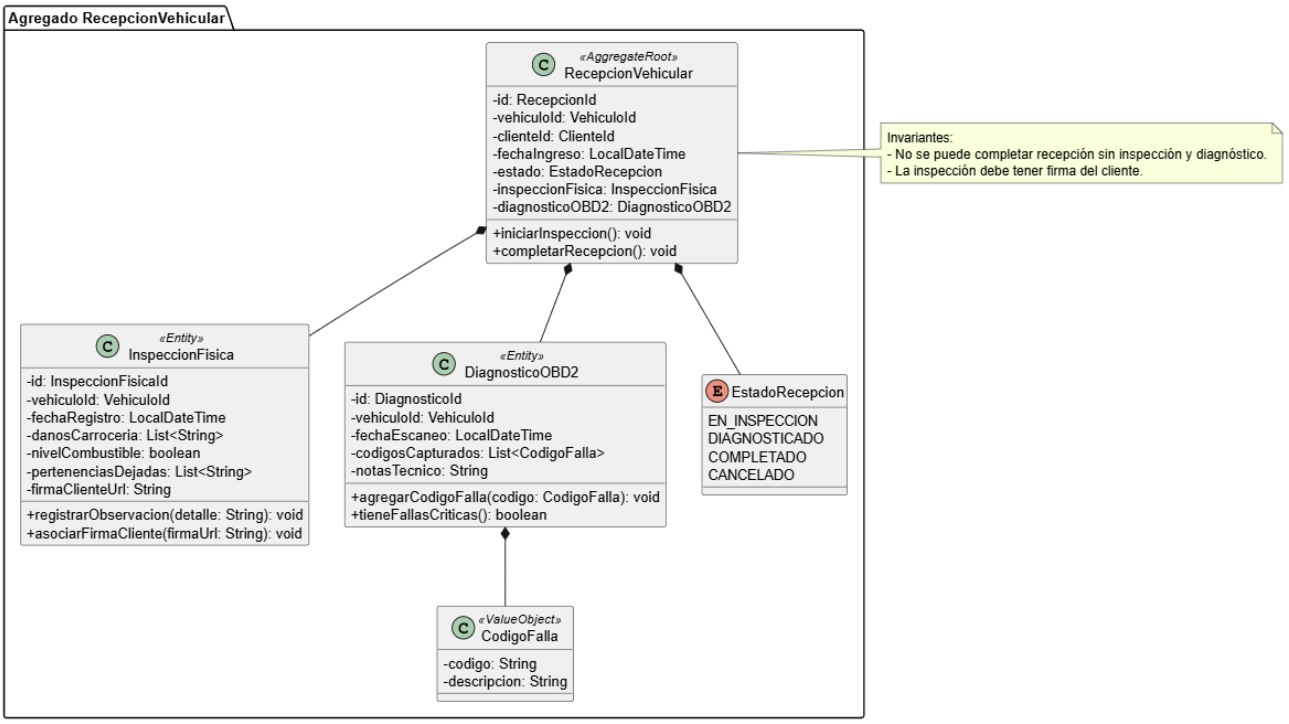
/**
 * 3. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos del dominio y sus reglas estructurales.
 */
enum EstadoRecepcion {
    EN_INSPECCION, DIAGNOSTICADO, COMPLETADO, CANCELADO
}

// Identificadores fuertemente tipados (encapsulan UUID)
record RecepcionId(UUID valor) {}
record VehiculoId(UUID valor) {}
record ClienteId(UUID valor) {}
record InspeccionFisicaId(UUID valor) {}
record DiagnosticoId(UUID valor) {}

// Objeto de valor con validación de formato
record CodigoFalla(String codigo, String descripcion) {
    /*
     * Regla de validación (Constructor compacto):
     * El 'codigo' debe coincidir con el patrón regex ^[PBCU]\d{4}$ (Ej:
P0104)
```

```
        */
}
```

Diagrama de Agregado



4.1.2. AGREGADO 3: Vehiculo

Entidad Raíz

Vehiculo

Propósito

Representa un vehículo que ingresa al taller y cuyo historial y características son fundamentales para la recepción y diagnóstico.

Invariantes / Reglas de Negocio Protegidas

El kilometraje no puede decrecer.

La placa debe tener un formato válido peruano.

El VIN no puede ser nulo ni vacío.

Entidades Internas

Nombre	Rol
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
VehiculoId	Vehiculo	Sí

VO	Usado por	Compartido
Placa	Vehiculo	No
Kilometraje	Vehiculo	No
ClienteId	Vehiculo	Sí

Atributos de la Entidad Raíz

Atributo	Tipo		Obligatorio	Descripción
id	VehiculoId		Sí	Identificador único del vehículo.
placa	Placa		Sí	Placa de rodaje del vehículo.
vin	String		Sí	Número de identificación vehicular.
marca	String		Sí	Marca del vehículo.
modelo	String		Sí	Modelo del vehículo.
kilometraje Actual	Kilometraje		Sí	Kilometraje registrado.
clienteId	ClienteId		Sí	Identificador del propietario.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Cliente	ClienteId	N:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
KilometrajeActualizado	Cuando se actualiza el kilometraje.
DatosPropietarioActualizados	Cuando cambian los datos de contacto del propietario.

Servicios de Dominio Relacionados

- ServicioEvaluacionGarantia: usa Vehiculo para calcular la garantía.

Codigo

```
/**
 * 1. AGREGADO RAÍZ
 * Representa el vehículo y gestiona sus características invariantes y
reglas principales.
 */
public class Vehiculo {
    // Atributos de identificación, valor y referencias a otros agregados
    private VehiculoId id;
    private Placa placa;
    private String vin;
    private String marca;
    private String modelo;
    private Kilometraje kilometrajeActual;
    private ClienteId clienteId;

    // --- Comportamientos del Agregado ---

    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta registrar un kilometraje menor al
actual
     * (el kilometraje de un vehículo nunca puede decrecer).
     */
    public void actualizarKilometraje(Kilometraje nuevoKilometraje) throws
IllegalArgumentException { /* ... */ }
}
/**
 * 2. OBJETOS DE VALOR (Value Objects)
 * Definen los tipos de datos propios del dominio y sus validaciones
estructurales (Inmutables).
 */

// Identificadores fuertemente tipados (encapsulan UUID)
```

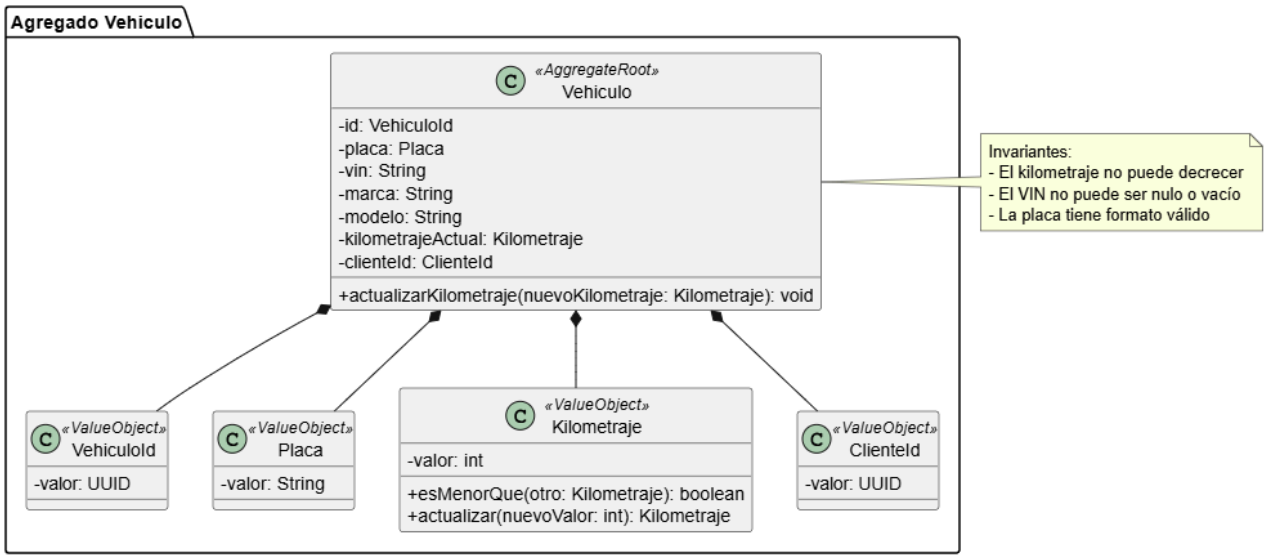
```java
record VehiculoId(UUID valor) {}
record ClienteId(UUID valor) {}

// Objeto de valor para la Placa
record Placa(String valor) {
    /*
     * Regla de validación (Constructor compacto):
     * Debe cumplir con los formatos válidos especificados por la expresión regular:
     * - 3 letras, guion, 3 números (Ej: ABC-123)
     * - 1 letra, 1 número, guion, 4 números (Ej: A1-1234)
     * - 2 letras, guion, 4 números (Ej: AB-1234)
     */
}
// Objeto de valor para el Kilometraje
record Kilometraje(int valor) {
    /*
     * Regla de validación (Constructor compacto):
     * El kilometraje no puede ser un número negativo (menor a 0).
     */

    // Retorna true si el valor actual es estrictamente menor que el de otro objeto Kilometraje
    public boolean esMenorQue(Kilometraje otro) { /* ... */ return false; }

    // Retorna una nueva instancia con el nuevo valor (respeta la inmutabilidad de los Value Objects)
    public Kilometraje actualizar(int nuevoValor) { /* ... */ return null;
}
}
```

Diagrama



4.1.3. AGREGADO 4: Cotizacion

Entidad Raíz

Cotizacion

Propósito

Representa la propuesta económica y de tiempo para las reparaciones necesarias, la cual debe ser aprobada por el cliente antes de proceder a la orden de trabajo.

Invariantes / Reglas de Negocio Protegidas

Requiere autorización explícita del cliente para ser aprobada (Regla de Negocio 1).
La cantidad de cada ítem debe ser > 0.
No se puede aprobar sin ítems en la cotización.

Entidades Internas

Nombre	Rol
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
CotizacionId	Cotizacion	Sí
VehiculoId	Cotizacion	Sí
EstadoCotizacion	Cotizacion	No
ItemCotizacion	Cotizacion	No
Monto	Cotizacion, ItemCotizacion	Sí
AutorizacionCliente	Cotizacion	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	CotizacionId	Sí	Identificador único de la cotización.
vehiculoId	VehiculoId	Sí	Vehículo asociado.
estado	EstadoCotizacion	Sí	Estado de la cotización.

Atributo	Tipo	Obligatorio	Descripción
items	List<ItemCotizacion>	Sí	Ítems cotizados.
totalEstimado	Monto	Sí	Costo total calculado.
diasInactividad Estimados	int	Sí	Tiempo estimado de trabajo.
fechaCreacion	LocalDateTime	Sí	Fecha de creación.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Vehiculo	VehiculoId	N:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
ItemAgregadoACotizacion	Al agregar un nuevo ítem a la cotización.
ItemRemovidoDeCotizacion	Al remover un ítem.
CotizacionAprobada	Al aprobar la cotización con autorización válida.

Servicios de Dominio Relacionados

- ServicioAprobacionCotizacion: orquesta la aprobación, reserva de stock y creación de la orden de trabajo.

.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Representa el vehículo y gestiona sus características invariantes y reglas principales.
 */
public class Vehiculo {
    // Atributos de identificación, datos base y referencias a otros agregados
    private VehiculoId id;
```

```
    private Placa placa;
    private String vin;
    private String marca;
    private String modelo;
    private Kilometraje kilometrajeActual;
    private ClienteId clienteId;

    // --- Comportamientos del Agregado ---

    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta registrar un kilometraje menor al
actual
     * (el kilometraje de un vehículo nunca puede decrecer).
     */
    public void actualizarKilometraje(Kilometraje nuevoKilometraje) throws
IllegalArgumentException { /* ... */ }
}


/**
 * 2. OBJETOS DE VALOR (Value Objects)
 * Definen los tipos de datos propios del dominio y sus validaciones
estructurales (Inmutables).
 */

// Identificadores fuertemente tipados (encapsulan UUID)
record VehiculoId(UUID valor) {}
record ClienteId(UUID valor) {}

// Objeto de valor para la Placa
record Placa(String valor) {
    /*
     * Regla de validación (Constructor compacto):
     * Debe cumplir con los formatos válidos especificados por la expresión
regular:
     * - 3 letras, guion, 3 números (Ej: ABC-123)
     * - 1 letra, 1 número, guion, 4 números (Ej: A1-1234)
     * - 2 letras, guion, 4 números (Ej: AB-1234)
     */
}

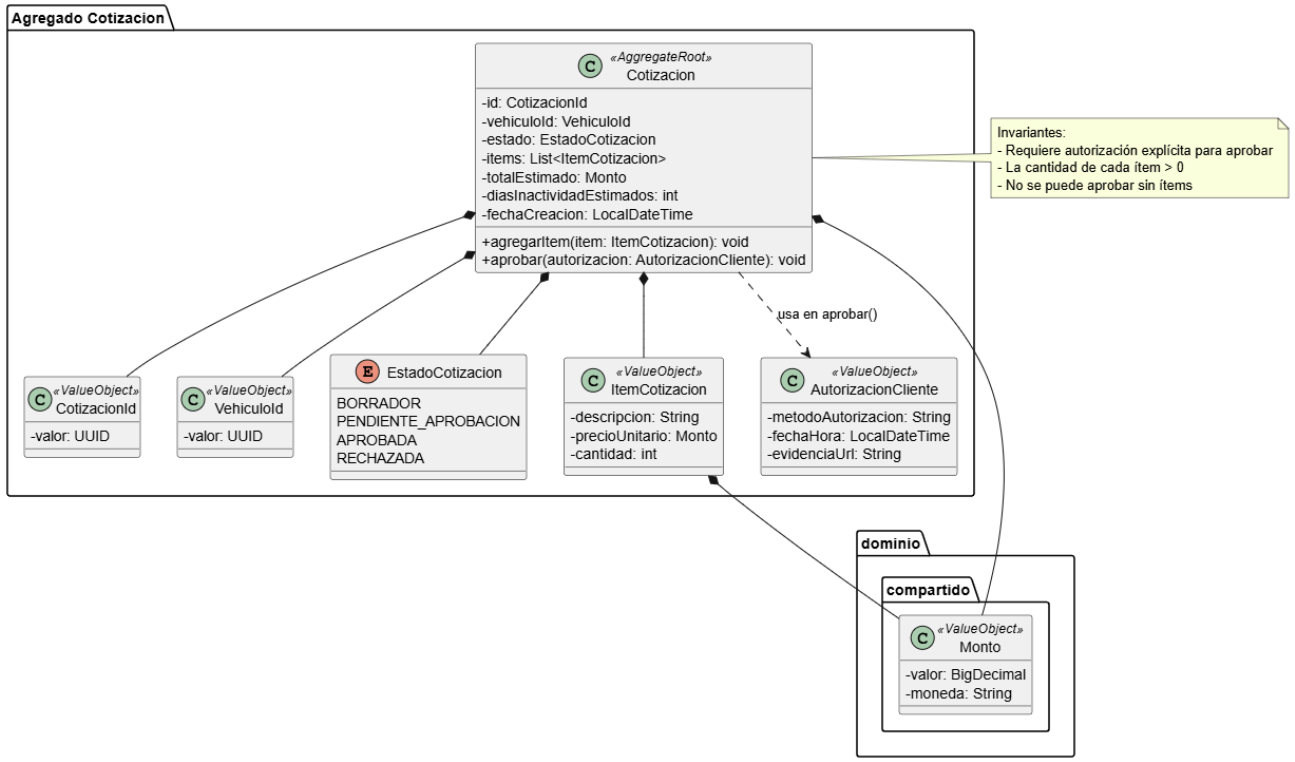
// Objeto de valor para el Kilometraje
record Kilometraje(int valor) {
    /*
     * Regla de validación (Constructor compacto):
```

```
     * El kilometraje no puede ser un número negativo (menor a 0).
     */

    // Retorna true si el valor actual es estrictamente menor que el de
otro objeto Kilometraje
    public boolean esMenorQue(Kilometraje otro) { /* ... */ return false; }

    // Retorna una nueva instancia con el nuevo valor (respeta la
inmutabilidad de los Value Objects)
    public Kilometraje actualizar(int nuevoValor) { /* ... */ return null;
}
}
```

Diagrama



4.2. CONTEXTO ACOTADO: OPERACIONES DE TALLER

4.2.1. AGREGADO 5: OrdenTrabajo

Entidad Raíz

OrdenTrabajo

Propósito

Gestionar la ejecución de las reparaciones de un vehículo en el taller, controlando tareas, repuestos y la asignación a una bahía y técnico.

Invariantes / Reglas de Negocio Protegidas

No se puede finalizar con tareas pendientes.

Las tareas requieren evidencia fotográfica para completarse.

No se cierra sin todas las tareas completadas.

Entidades Internas

Nombre	Rol
TareaReparacion	Registra el progreso y evidencias de una reparación específica.

Objetos de Valor Utilizados

VO	Usado por	Compartido
OrdenTrabajoId	OrdenTrabajo	No
TareaReparacionId	TareaReparacion	No
FotoEvidencia	TareaReparacion	No
RegistroUsoRepuesto	TareaReparacion	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	OrdenTrabajoId	Sí	Identificador único.
cotizacionId	CotizacionId	Sí	Referencia a la cotización aprobada.
vehiculoId	VehiculoId	Sí	Referencia al vehículo.
tecnicoAsignadoId	TecnicoId	Sí	Referencia al técnico.
bahiaId	BahiaTrabajoId	Sí	Referencia a la bahía asignada.
estado	EstadoOrdenTrabajo	Sí	Estado de la orden.
tareas	List<TareaReparacion>	Sí	Lista de tareas.
fechaInicio	LocalDateTime	No	Fecha de inicio real.
fechaFinEstim	LocalDateTime	No	Fecha fin estimada.

Atributo	Tipo	Obligatorio	Descripción
ada			

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Cotizacion	CotizacionId	1:1
Vehiculo	VehiculoId	N:1
Tecnico	TecnicoId	N:1
BahiaTrabajo	BahiaTrabajoId	N:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
OrdenTrabajoCreada	Creación de la orden.
BahiaAsignadaAOrden	Asignación de bahía.
OrdenBloqueadaPorFaltaRepuesto	Bloqueo por falta de repuestos.
TareaReparacionCompletada	Completar una tarea.
OrdenTrabajoFinalizada	Marcar lista para calidad.

Servicios de Dominio Relacionados

- ServicioAprobacionCotizacion: Crea la OrdenTrabajo.
- ServicioAsignacionTaller: Asigna bahía y técnico.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona el ciclo de vida de la ejecución del servicio y coordina las
tareas de reparación.
 */
public class OrdenTrabajo {
    // Identificadores y referencias a otros agregados
```

```
    private OrdenTrabajoId id;
    private CotizacionId cotizacionId;
    private VehiculoId vehiculoId;
    private TecnicoId tecnicoAsignadoId;
    private BahiaTrabajoId bahiaId;

    // Estado y control de tiempo
    private EstadoOrdenTrabajo estado;
    private LocalDateTime fechaInicio;
    private LocalDateTime fechaFinEstimada;

    // Entidades hijas
    private List<TareaReparacion> tareas;

    // --- Comportamientos del Agregado ---

    // Asigna la bahía, cambia el estado a EN_PROCESO y registra la fecha
de inicio
    public void asignarBahia(BahiaTrabajoId bahiaId) { /* ... */ }

    /*
     * Regla de negocio:
     * Lanza excepción si se intenta bloquear una orden que ya está en
estado FINALIZADO.
     */
    public void bloquearPorFaltaRepuesto(String motivo) throws
IllegalStateException { /* ... */ }

    // Delega la acción de completar a la tarea específica correspondiente
    public void completarTarea(TareaReparacionId tareaId) { /* ... */ }

    /*
     * Regla de negocio estricta:
     * No se puede marcar la orden como lista para calidad (FINALIZADO)
     * si existe al menos una tarea que no esté en estado COMPLETADA.
     */
    public void marcarComoListaParaCalidad() throws IllegalStateException {
/* ... */ }
}


/**
 * 2. ENTIDADES INTERNAS
 * Representan los trabajos individuales a realizar dentro de la orden.
 */
class TareaReparacion {
```

```
    private TareaReparacionId id;
    private String descripcion;
    private EstadoTarea estado;
    private List<FotoEvidencia> evidencias;
    private List<RegistroUsoRepuesto> repuestosUtilizados;

    public void adjuntarEvidenciaFotografica(FotoEvidencia evidencia) { /*
... */ }

    public void registrarConsumoRepuesto(RegistroUsoRepuesto uso) { /* ...
*/ }

    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta completar la tarea sin haber adjuntado
     * al menos una evidencia fotográfica.
     */
    public void completar() throws IllegalStateException { /* ... */ }
}



/**
 * 3. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos inmutables y reglas estructurales del dominio.
 */
enum EstadoOrdenTrabajo {
    EN_ESPERA, EN_PROCESO, BLOQUEADO_POR_REPUESTO, FINALIZADO
}

enum EstadoTarea {
    PENDIENTE, EN_EJECUCION, COMPLETADA
}

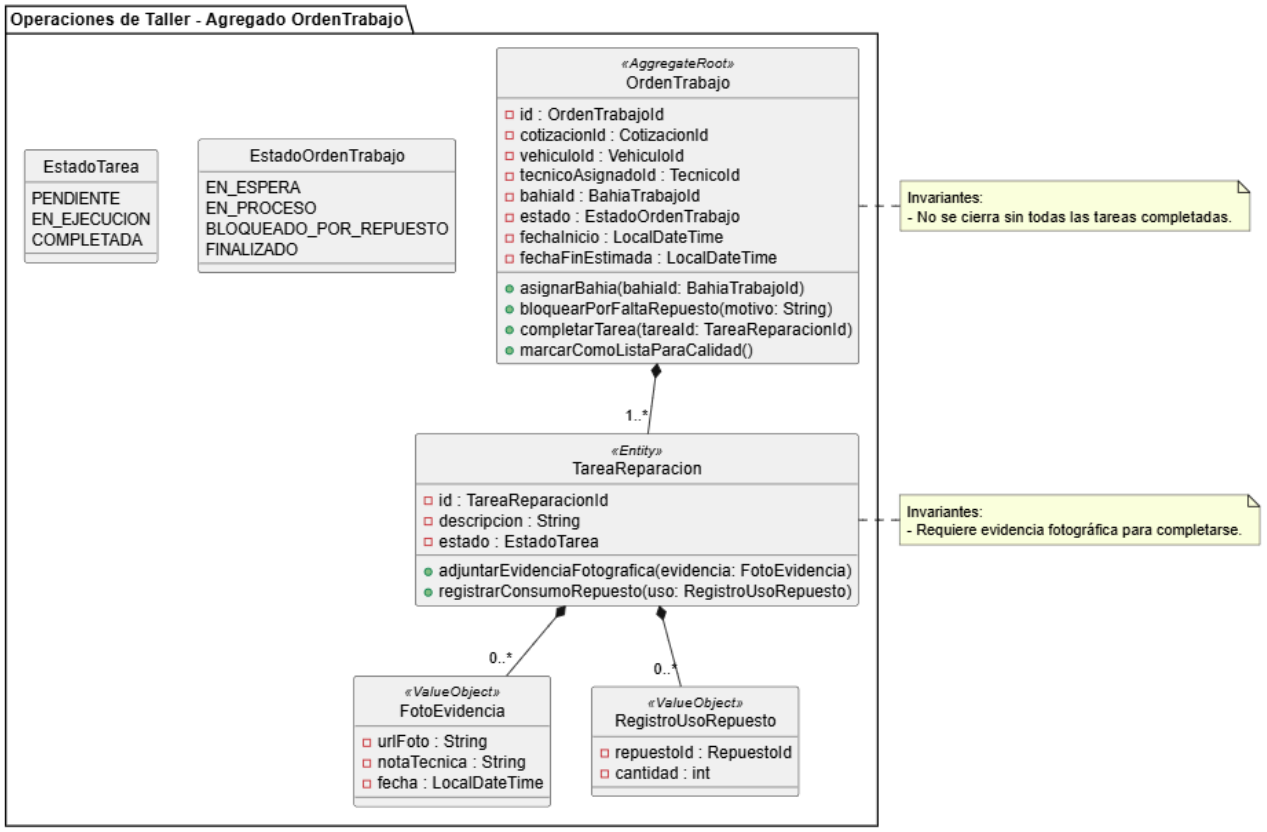
// Identificadores fuertemente tipados
record OrdenTrabajoId(UUID valor) {}
record TareaReparacionId(UUID valor) {}
record CotizacionId(UUID valor) {}
record VehiculoId(UUID valor) {}
record TecnicoId(UUID valor) {}
record BahiaTrabajoId(UUID valor) {}
record RepuestoId(UUID valor) {}

// Objetos de valor con reglas de validación
record FotoEvidencia(String urlFoto, String notaTecnica, LocalDateTime
fecha) {
    /*
```

```
     * Regla de validación (Constructor compacto):
     * La 'urlFoto' no puede ser nula ni estar vacía. La 'fecha' es
obligatoria.
     */
}

record RegistroUsoRepuesto(RepuestoId repuestoId, int cantidad) {
    /*
     * Regla de validación (Constructor compacto):
     * El 'repuestoId' es obligatorio y la 'cantidad' debe ser mayor a 0.
     */
}
```

Diagrama



4.2.2. AGREGADO 6: BahiaTrabajo

Entidad Raíz

BahiaTrabajo

Propósito

Gestionar la disponibilidad y asignación de las bahías de trabajo dentro del taller.

Invariantes / Reglas de Negocio Protegidas

No se puede ocupar si no está LIBRE.

No se puede liberar si no está OCUPADA.

Entidades Internas

Ninguna

Objetos de Valor Utilizados

VO	Usado por	Compartido
BahiaTrabajoId	BahiaTrabajo	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	BahiaTrabajoId	Sí	Identificador único.
codigo	String	Sí	Código legible de la bahía (ej. BAHIA-01).
tipo	TipoBahia	Sí	Tipo de bahía.
estado	EstadoBahia	Sí	Estado de ocupación.

Referencias a Otros Agregados

Ninguna directa almacenada (OrdenTrabajo referencia BahiaTrabajo por BahiaTrabajoId)

Eventos de Dominio Publicados

Evento	Acción que lo dispara
BahiaOcupada	Al ocupar la bahía con una orden de trabajo.
BahiaLiberada	Al liberar la bahía.
BahiaPuestaEnMantenimiento	Al poner la bahía en mantenimiento.

Servicios de Dominio Relacionados

- ServicioAsignacionTaller: Valida bahía libre y la ocupa.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona el estado y disponibilidad de las bahías físicas del taller.
 */
public class BahiaTrabajo {
    // Atributos de identificación y características
    private BahiaTrabajoId id;
    private String codigo;
    private TipoBahia tipo;
```

```
    // Estado actual de la bahía
    private EstadoBahia estado;

    // --- Comportamientos del Agregado ---

    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta asignar una orden a una bahía
     * cuyo estado no sea exactamente LIBRE. Cambia el estado a OCUPADA.
     */
    public void ocuparConOrden(OrdenTrabajoId ordenId) throws
IllegalStateException { /* ... */ }

    /*
     * Regla de negocio:
     * Lanza excepción si se intenta liberar una bahía que no esté OCUPADA.
     * Revierte el estado a LIBRE.
     */
    public void liberar() throws IllegalStateException { /* ... */ }

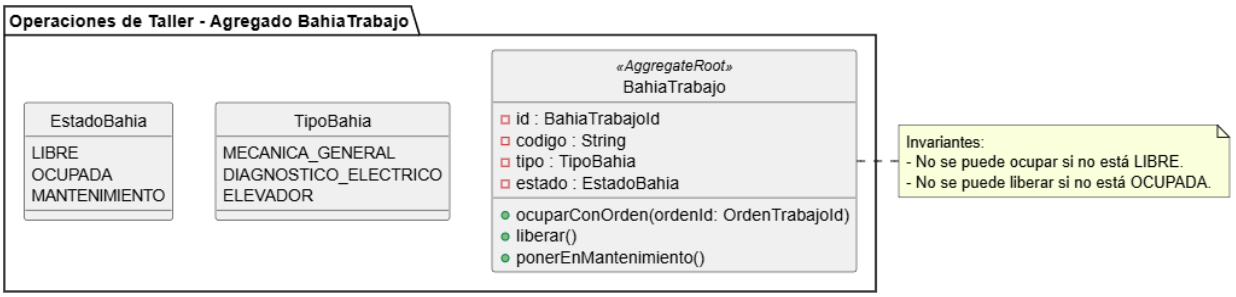
    // Pone la bahía fuera de servicio temporalmente (MANTENIMIENTO)
    public void ponerEnMantenimiento() { /* ... */ }
}


/**
 * 2. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos inmutables y estados permitidos dentro del dominio.
 */
enum TipoBahia {
    MECANICA_GENERAL, DIAGNOSTICO_ELECTRICO, ELEVADOR
}

enum EstadoBahia {
    LIBRE, OCUPADA, MANTENIMIENTO
}

// Identificadores fuertemente tipados
record BahiaTrabajoId(UUID valor) {}
record OrdenTrabajoId(UUID valor) {}
```

Diagrama



4.3. CONTEXTO ACOTADO: CONTROL DE CALIDAD Y ENTREGA

4.3.1. AGREGADO 7: PruebaCalidad

Entidad Raíz

PruebaCalidad

Propósito

Asegurar que el vehículo reparado cumple con los estándares de calidad antes de la entrega al cliente.

Invariantes / Reglas de Negocio Protegidas

Requiere 100% de checks de seguridad aprobados para emitir dictamen.

No se puede aprobar sin prueba estática y de ruta completadas y exitosas.

Entidades Internas

Ninguna

Objetos de Valor Utilizados

VO	Usado por	Compartido
PruebaCalidadId	PruebaCalidad	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	PruebaCalidadId	Sí	Identificador único de la prueba.
ordenTrabajoId	OrdenTrabajoId	Sí	Referencia a la orden de trabajo.
supervisorId	SupervisorCalidadId	Sí	Referencia externa al supervisor.
apruebaPruebaEstatica	boolean	Sí	Resultado estático.
apruebaPruebaRuta	boolean	Sí	Resultado de la ruta.

Atributo	Tipo	Obligatorio	Descripción
estado	EstadoPruebaCalidad	Sí	Estado del control.
observaciones	String	No	Observaciones o motivos de rechazo.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
OrdenTrabajo	OrdenTrabajoId	1:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
ResultadosEstaticosRegistrados	Al registrar la prueba estática.
ResultadosRutaRegistrados	Al registrar la prueba de ruta.
PruebaCalidadAprobada	Al aprobar todo el control.
PruebaCalidadRechazada	Al enviar a retrabajo.

Servicios de Dominio Relacionados

Ninguno directo.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona el proceso de inspección, verificación del trabajo y decisión
final sobre la reparación.
 */
public class PruebaCalidad {
    // Identificadores y referencias
    private PruebaCalidadId id;
    private OrdenTrabajoId ordenTrabajoId;
    private SupervisorCalidadId supervisorId;

    // Resultados de las validaciones y estado general
    private boolean apruebaPruebaEstatica;
    private boolean apruebaPruebaRuta;
```

```
    private EstadoPruebaCalidad estado;
    private String observaciones;

    // --- Comportamientos del Agregado ---

    // Registra el resultado de la revisión estática y acumula detalles si
hay fallos
    public void registrarResultadoEstatico(boolean exitoso, String
detalles) { /* ... */ }

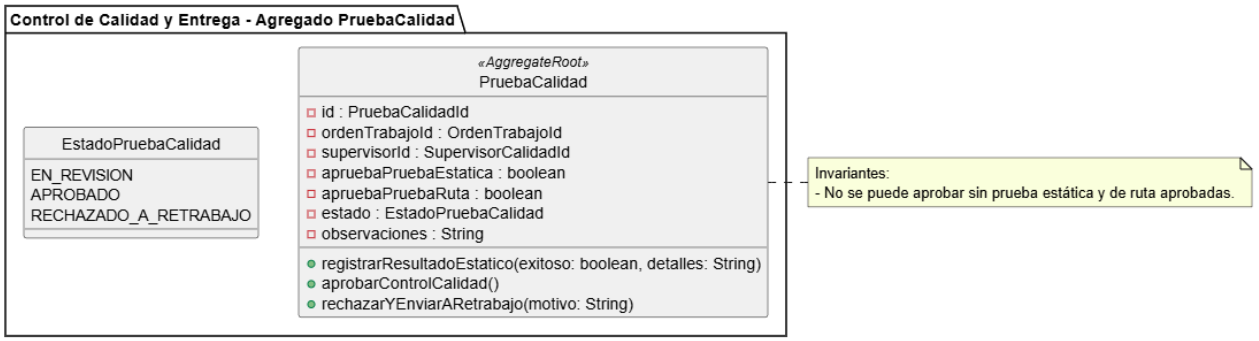
    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta aprobar el control de calidad
(APROBADO)
     * sin que la prueba estática haya sido marcada como exitosa
previamente.
     */
    public void aprobarControlCalidad() throws IllegalStateException { /*
... */ }

    /*
     * Regla de negocio:
     * Lanza excepción si no se proporciona un motivo válido.
     * Cambia el estado a RECHAZADO_A_RETRABAJO y guarda la observación.
     */
    public void rechazarYEnviarARetrabajo(String motivo) throws
IllegalArgumentException { /* ... */ }
}


/**
 * 2. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos inmutables y estados permitidos dentro del dominio.
 */
enum EstadoPruebaCalidad {
    EN_REVISION, APROBADO, RECHAZADO_A_RETRABAJO
}

// Identificadores fuertemente tipados (encapsulan UUID)
record PruebaCalidadId(UUID valor) {}
record OrdenTrabajoId(UUID valor) {}
record SupervisorCalidadId(UUID valor) {}
```

Diagrama



4.3.2. AGREGADO 8: InformeEntrega

Entidad Raíz

InformeEntrega

Propósito

Consolidar los datos finales de la reparación, incluyendo la garantía otorgada, recomendaciones futuras de mantenimiento y constancia de lavado, para la entrega final al cliente.

Invariantes / Reglas de Negocio Protegidas

La garantía es inmutable: siempre se otorgan 3 meses o 5,000 km (Regla de Negocio 3).

El informe requiere obligatoriamente una orden de trabajo asociada.

Entidades Internas

Ninguna

Objetos de Valor Utilizados

VO	Usado por	Compartido
InformeEntregaId	InformeEntrega	No
CondicionGarantia	InformeEntrega	No
RegistroMantenimientoSugerido	InformeEntrega	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	InformeEntregaId	Sí	Identificador único.
ordenTrabajoId	OrdenTrabajoId	Sí	Referencia a la orden finalizada.

Atributo	Tipo	Obligatorio	Descripción
fechaEntrega	LocalDateTime	Sí	Fecha de entrega.
garantiaOtorgada	CondicionGarantia	Sí	Garantía de la reparación.
proximoMantenimiento	RegistroMantenimientoSugerido	No	Sugerencia próximo servicio.
vehiculoLavado	boolean	Sí	Confirmación de lavado del vehículo.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
OrdenTrabajo	OrdenTrabajoId	1:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
InformeEntregaGenerado	Creación del informe de entrega.
ProximoMantenimientoProgramado	Al registrar el mantenimiento sugerido.

Servicios de Dominio Relacionados

- ServicioEvaluacionGarantia: Usa InformeEntrega para validar cobertura activa.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona la entrega final del vehículo al cliente, estableciendo las políticas
 * de garantía y sugerencias de mantenimiento futuro.
 */
public class InformeEntrega {
    // Identificadores y referencias
    private InformeEntregaId id;
    private OrdenTrabajoId ordenTrabajoId;

    // Datos de la entrega y condiciones
```

```
    private LocalDateTime fechaEntrega;
    private CondicionGarantia garantiaOtorgada;
    private RegistroMantenimientoSugerido proximoMantenimiento;
    private boolean vehiculoLavado;

    // --- Comportamientos del Agregado ---

    /*
     * Nota de regla de negocio (Constructor):
     * Al crearse el Informe de Entrega, se asigna automáticamente la
garantía aplicando
     * la política inmutable del taller: exactamente 3 meses y 5000 km
desde la fecha de entrega.
     */

    // Asigna el registro calculado para la próxima visita preventiva del
cliente
    public void programarProximoMantenimiento(int kilometrajeSugerido,
LocalDateTime fechaEstimada) { /* ... */ }
}



/**
 * 2. OBJETOS DE VALOR (Value Objects)
 * Tipos de datos inmutables y reglas estructurales del dominio.
 */

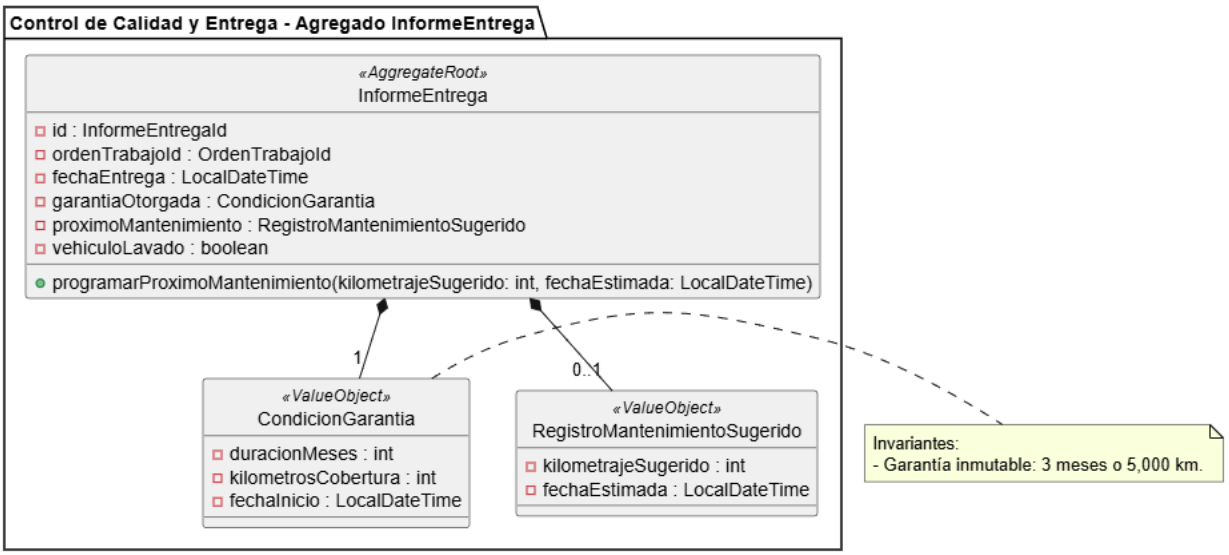
// Identificadores fuertemente tipados (encapsulan UUID)
record InformeEntregaId(UUID valor) {}
record OrdenTrabajoId(UUID valor) {}

// Objeto de valor para las condiciones de la garantía
record CondicionGarantia(int duracionMeses, int kilometrosCobertura,
LocalDateTime fechaInicio) {
    /*
     * Regla de validación estricta (Constructor compacto):
     * La política del taller exige que la garantía ingresada sea
obligatoriamente
     * de 3 meses (duracionMeses) y 5000 km (kilometrosCobertura).
     */
}

// Objeto de valor para la programación del siguiente servicio
record RegistroMantenimientoSugerido(int kilometrajeSugerido, LocalDateTime
fechaEstimada) {
    /*
```

```
     * Regla de validación (Constructor compacto):
     * El 'kilometrajeSugerido' debe ser mayor a 0 y la 'fechaEstimada' no
puede ser nula.
     */
}
```

Diagrama



4.4. CONTEXTO ACOTADO: INVENTARIO Y ALMACÉN

4.4.3. AGREGADO 9: Repuesto

Entidad Raíz

Repuesto

Propósito

Gestionar la disponibilidad, los detalles y el inventario de las piezas (repuestos) utilizadas durante los mantenimientos o reparaciones.

Invariantes / Reglas de Negocio Protegidas

El stock reservado no puede exceder el stock físico real.

No se puede despachar más cantidad de la que se tiene disponible.

Si la diferencia entre el stock físico real y el stock reservado es menor que la cantidad mínima del umbral, se debe disparar una alerta (alcanza el umbral crítico).

Entidades Internas

Nombre	**Rol**
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
RepuestoId	Identidad de Repuesto	No
UmbralReposicion	Límite de reposición	No
Monto	Precio Unitario	Sí

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	RepuestoId	Sí	Identificador único del repuesto.
codigoSku	String	Sí	Código de inventario único.
nombre	String	Sí	Nombre o descripción del repuesto.
stockFisicoReal	int	Sí	Cantidad física en almacén.
stockReservado	int	Sí	Cantidad apartada para cotizaciones.
umbral	UmbralReposicion	Sí	Umbral que define cuándo reponer.
precioUnitario	Monto	Sí	Precio unitario del repuesto.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Ninguna	-	-

Eventos de Dominio Publicados

Evento	Acción que lo dispara
StockRepuestoAgregado	Al incrementar el inventario.
StockRepuestoDeducido	Al despachar repuestos o mermar.
UmbralCriticoAlcanzado	Al detectarse que el stock disponible cae por debajo de la cantidad mínima del umbral.

Servicios de Dominio Relacionados

- ServicioAprobacionCotizacion: reserva stock en Repuesto.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona la información de inventario y el ciclo de vida del stock de repuestos.
 */
public class Repuesto {
    // Identificadores y datos básicos
    private RepuestoId id;
    private String codigoSku;
    private String nombre;
    private Monto precioUnitario;

    // Control de inventario
    private int stockFisicoReal;
    private int stockReservado;
    private UmbralReposicion umbral;

    // --- Comportamientos del Agregado ---

    /*
     * Nota de regla de negocio (Invariante estructural):
     * El stock reservado nunca puede ser mayor al stock físico real.
     */

    /*
     * Regla de negocio estricta:
     * Lanza excepción si la cantidad a reservar supera el stock disponible
     * (es decir, si lo ya reservado + la nueva cantidad supera el stock físico real).
     */
```

```
    public void reservarStock(int cantidad) throws IllegalStateException,
IllegalArgumentException { /* ... */ }

    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta despachar más stock del físico
existente.
     * Al despachar, reduce el stock físico real y libera el stock
reservado proporcionalmente.
     */
    public void despacharStock(int cantidad) throws IllegalStateException,
IllegalArgumentException { /* ... */ }

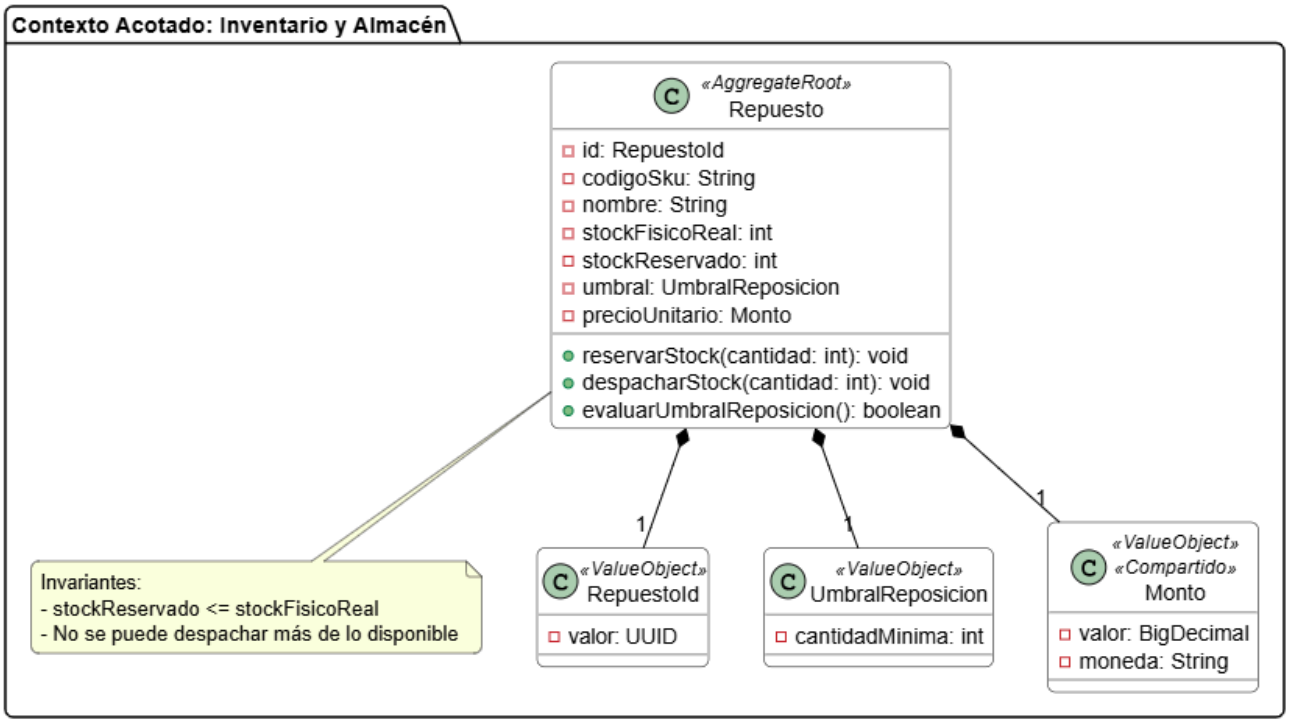
    /*
     * Regla de negocio:
     * Retorna true si el stock disponible (stock físico real menos stock
reservado)
     * es menor que la cantidad mínima definida en el umbral de reposición.
     */
    public boolean evaluarUmbralReposicion() { /* ... */ return false; }
}


/**
 * 2. OBJETOS DE VALOR (Value Objects)
 * Tipos de datos inmutables y reglas estructurales del dominio.
 */

// Identificador fuertemente tipado (encapsula UUID)
record RepuestoId(UUID valor) {}

// Objeto de valor para definir las alertas de stock
record UmbralReposicion(int cantidadMinima) {
    /*
     * Regla de validación (Constructor compacto):
     * La cantidad mínima de reposición no puede ser un valor negativo.
     */
}
```

Diagrama



4.4.4. AGREGADO 10: ReservaRepuesto

Entidad Raíz

ReservaRepuesto

Propósito

Representa el compromiso de separar piezas del inventario en base a una cotización, con vigencia temporal.

Invariantes / Reglas de Negocio Protegidas

La reserva requiere estar vinculada obligatoriamente a un CotizacionId.

Posee una fecha de expiración límite.

Solo puede ser consumida si actualmente se encuentra en estado ACTIVA.

Entidades Internas

Nombre	Rol
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
ReservaId	Identidad de ReservaRepuesto	No
CotizacionId	Referencia externa	No
RepuestoId	Referencia externa	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	ReservaId	Sí	Identificador único de la reserva.
cotizacionId	CotizacionId	Sí	Referencia a la Cotización asociada.
repuestoId	RepuestoId	Sí	Referencia al Repuesto que se reserva.
cantidad	int	Sí	Cantidad del repuesto a reservar.
estado	EstadoReserva	Sí	Estado de la reserva (ACTIVA, CONSUMIDA, CANCELADA).
fechaExpiracion	LocalDateTime	Sí	Fecha tope antes de expirar la reserva.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Cotizacion	CotizacionId	N:1
Repuesto	RepuestoId	N:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
ReservaRepuestoConfirmada	Al concretarse la separación de stock.
ReservaRepuestoConsumida	Al consumirse los repuestos en un servicio.
ReservaRepuestoExpirada	Al pasar la fechaExpiracion sin consumirse.

Servicios de Dominio Relacionados

- ServicioAprobacionCotizacion: crea la ReservaRepuesto al aprobar la cotización.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona el ciclo de vida temporal de la reserva de un repuesto
vinculado a una cotización.
 */
public class ReservaRepuesto {
    // Identificadores y referencias a otros agregados
    private ReservaId id;
    private CotizacionId cotizacionId;
    private RepuestoId repuestoId;

    // Detalles operativos y estado
    private int cantidad; // Por regla (en creación), siempre debe ser
mayor a 0
    private EstadoReserva estado;
    private LocalDateTime fechaExpiracion;

    // --- Comportamientos del Agregado ---

    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta consumir una reserva cuyo estado no
sea exactamente ACTIVA.
     * Si es válida, cambia el estado a CONSUMIDA.
     */
    public void marcarComoConsumida() throws IllegalStateException { /* ...
*/ }

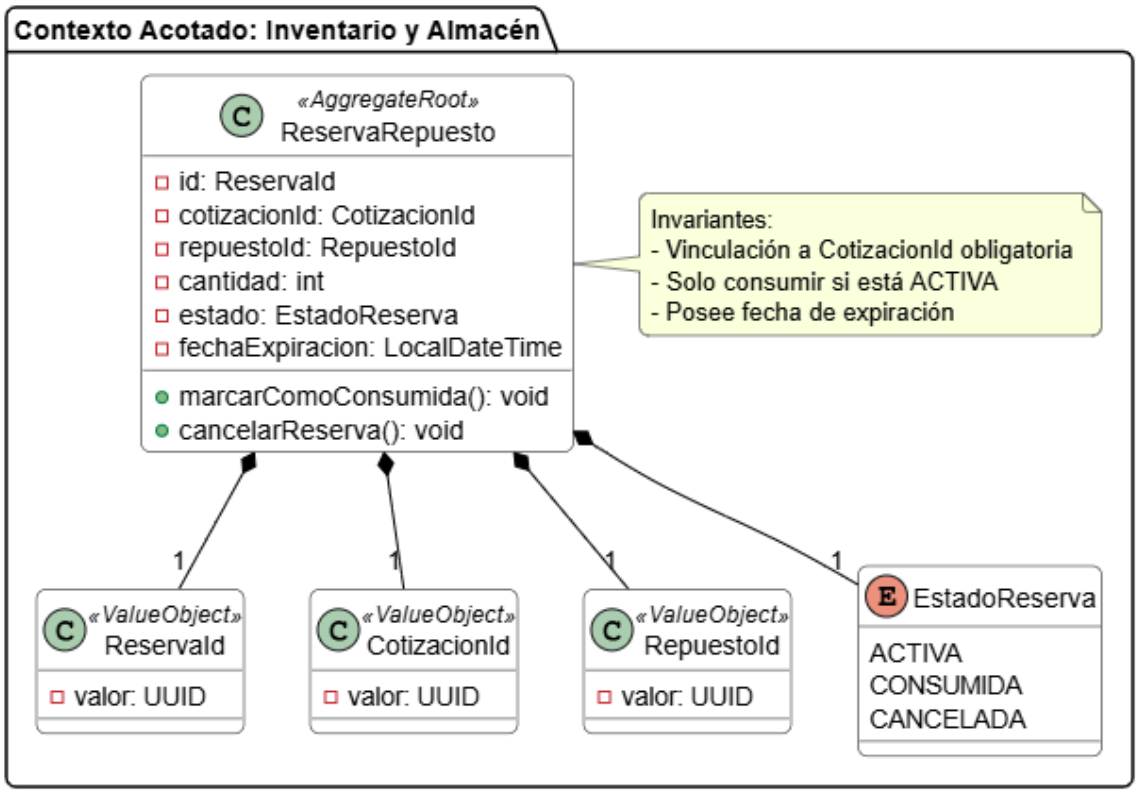
    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta cancelar una reserva cuyo estado no
sea exactamente ACTIVA.
     * Si es válida, cambia el estado a CANCELADA.
     */
    public void cancelarReserva() throws IllegalStateException { /* ... */
}
}


/**
 * 2. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos inmutables y estados permitidos dentro del dominio.
 */
```

```
enum EstadoReserva {
    ACTIVA, CONSUMIDA, CANCELADA
}

// Identificadores fuertemente tipados (encapsulan UUID)
record ReservaId(UUID valor) {}
record CotizacionId(UUID valor) {}
record RepuestoId(UUID valor) {}
```

Diagrama



4.5. CONTEXTO ACOTADO: AGENDAMIENTO

4.5.5. AGREGADO 11: Cita

Entidad Raíz

Cita

Propósito

Programar y gestionar los turnos de atención para los clientes y sus vehículos en el taller.

Invariantes / Reglas de Negocio Protegidas

No se puede programar una cita con fecha en el pasado.

El slot horario debe ser válido (la hora de inicio debe ser estrictamente anterior a la hora de fin).

Entidades Internas

Nombre	Rol
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
CitaId	Identidad de Cita	No
ClienteId	Referencia externa	No
VehiculoId	Referencia externa	No
SlotHorario	Bloque de tiempo	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	CitaId	Sí	Identificador de la cita.
clienteId	ClienteId	Sí	Referencia al cliente.
vehiculoId	VehiculoId	Sí	Referencia al vehículo asociado.
fechaProgramada	LocalDateTime	Sí	Fecha y hora en la que se atenderá.
bloqueHorario	SlotHorario	Sí	Duración o ventana del turno.
estado	EstadoCita	Sí	Estado de la cita.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Cliente	ClienteId	N:1
Vehiculo	VehiculoId	N:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
CitaProgramada	Al crear la cita correctamente.
CitaReprogramada	Al modificar la fecha o slot horario.
CitaCancelada	Al anular la cita.

Servicios de Dominio Relacionados

Ninguno.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona la programación, estado y ciclo de vida de las citas para la atención de vehículos.
 */
public class Cita {
    // Identificadores y referencias a otros agregados
    private CitaId id;
    private ClienteId clienteId;
    private VehiculoId vehiculoId;

    // Datos de programación y estado temporal
    private LocalDateTime fechaProgramada;
    private SlotHorario bloqueHorario;
    private EstadoCita estado;

    // --- Comportamientos del Agregado ---

    /*
     * Nota de regla de negocio (Constructor / Creación):
     * Lanza excepción si se intenta programar una cita en el pasado (antes del momento actual).
     * Toda cita nueva inicia por defecto en estado PROGRAMADA.
```

```
     */

    // Cambia el estado de la cita a CONFIRMADA
    public void confirmarCita() { /* ... */ }

    // Cambia el estado de la cita a ATENDIDA una vez el vehículo ingresa
al taller
    public void marcarComoAtendida() { /* ... */ }

    // Cambia el estado de la cita a CANCELADA
    public void cancelar() { /* ... */ }

    /*
     * Regla de negocio estricta:
     * Lanza excepción si se intenta establecer una nueva fecha que esté en
el pasado.
     * Actualiza la fecha, el bloque horario y revierte el estado a
PROGRAMADA.
     */
    public void reprogramar(LocalDateTime nuevaFecha, SlotHorario
nuevoBloque) throws IllegalArgumentException { /* ... */ }
}

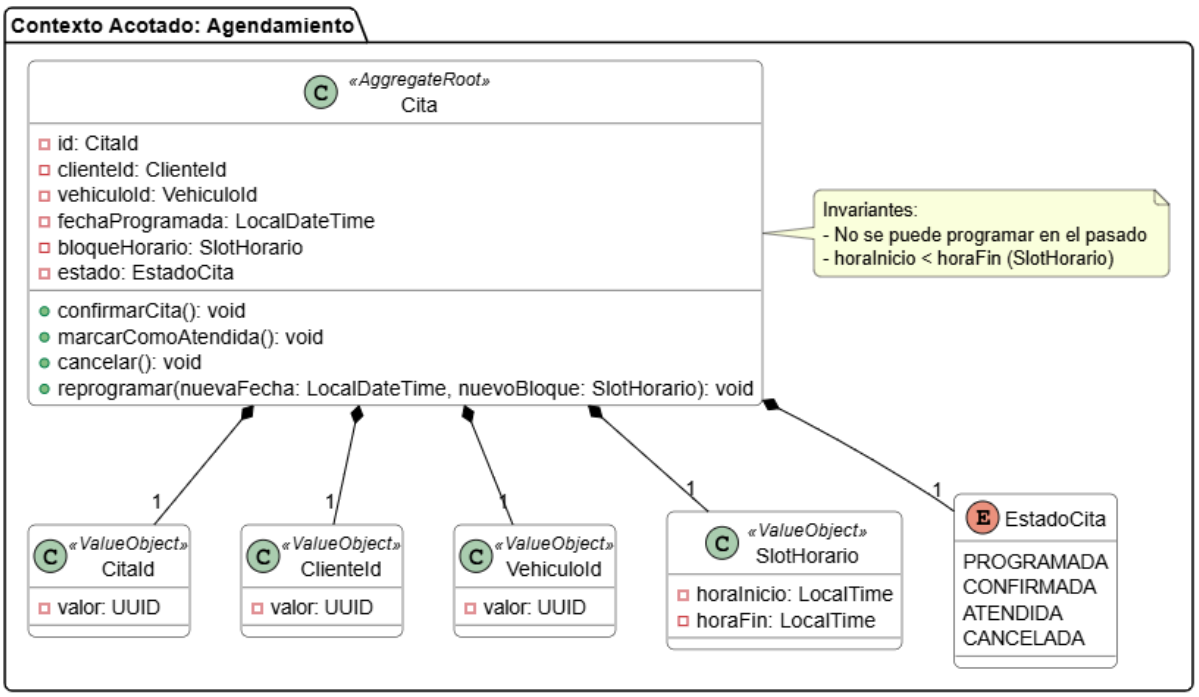

/**
 * 2. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos inmutables y reglas estructurales del dominio temporal.
 */

enum EstadoCita {
    PROGRAMADA, CONFIRMADA, ATENDIDA, CANCELADA
}

// Identificadores fuertemente tipados (encapsulan UUID)
record CitaId(UUID valor) {}
record ClienteId(UUID valor) {}
record VehiculoId(UUID valor) {}

// Objeto de valor para representar la duración agendada (Bloque temporal)
record SlotHorario(LocalTime horaInicio, LocalTime horaFin) {
    /*
     * Regla de validación estricta (Constructor compacto):
     * La 'horaInicio' debe ser cronológicamente anterior (menor) a la
'horaFin'.
     */
}
```

Diagrama



4.6. CONTEXTO ACOTADO: GESTIÓN DE PERSONAL

4.6.6. AGREGADO 12: Tecnico

Entidad Raíz

Tecnico

Propósito

Controlar la disponibilidad, los turnos y especialidades de los técnicos del taller, para permitir la correcta asignación de trabajos.

Invariantes / Reglas de Negocio Protegidas

No se puede asignar a un técnico si su estado actual no es DISPONIBLE.

No se deben asignar tareas al técnico fuera de su turno laboral.

Entidades Internas

Nombre	Rol
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
TecnicoId	Identidad de Tecnico	No
TurnoHorario	Horas de labor	No

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	TecnicoId	Sí	Identificador del técnico.
dni	String	Sí	Documento de identidad.
nombreCompleto	String	Sí	Nombre del empleado.
especialidades	List<Especialidad>	Sí	Lista de ramas que domina.
estado	EstadoDisponibilidad	Sí	Condición actual.
turnoLaboral	TurnoHorario	Sí	Jornada de trabajo del técnico.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
Ninguna	-	-

Eventos de Dominio Publicados

Evento	Acción que lo dispara
TurnoTecnicoActualizado	Al modificar su horario de trabajo.
TecnicoEnDescanso	Al cambiar el estado a descanso.
TecnicoDisponible	Al liberar el técnico tras acabar trabajo.

Servicios de Dominio Relacionados

- ServicioAsignacionTaller: Valida disponibilidad y especialidad del técnico antes de asignar la orden.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona la disponibilidad, especialidades y asignación de los técnicos
a las órdenes de trabajo.
 */
public class Tecnico {
    // Identificadores y datos personales
    private TecnicoId id;
    private String dni;
    private String nombreCompleto;

    // Atributos de negocio y control de disponibilidad
    private List<Especialidad> especialidades;
    private EstadoDisponibilidad estado;
    private TurnoHorario turnoLaboral;

    // --- Comportamientos del Agregado ---

    /*
     * Nota de regla de negocio (Constructor):
     * Un técnico debe tener obligatoriamente al menos una especialidad
registrada al crearse.
     * Su estado inicial por defecto siempre es DISPONIBLE.
     */

    /*
     * Regla de negocio estricta:
     * Lanza excepción si el estado actual NO es DISPONIBLE o si el
'momento'
     * en el que se le intenta asignar cae fuera de su turno laboral
(TurnoHorario).
     * Si pasa las validaciones, cambia el estado a OCUPADO.
     */
    public void asignarAOrden(LocalDateTime momento) throws
IllegalStateException { /* ... */ }

    // Libera al técnico de su asignación, volviendo su estado a DISPONIBLE
    public void liberarTecnico() { /* ... */ }

    // Cambia el estado del técnico a EN_DESCANSO (pausas, almuerzos, etc.)
    public void ponerEnDescanso() { /* ... */ }
}


/**
```

```
 * 2. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos inmutables y reglas estructurales del dominio del
personal.
 */

enum Especialidad {
    MECANICA_LIGERA, DIAGNOSTICO_OBD2, FRENOS, ELECTRICIDAD
}

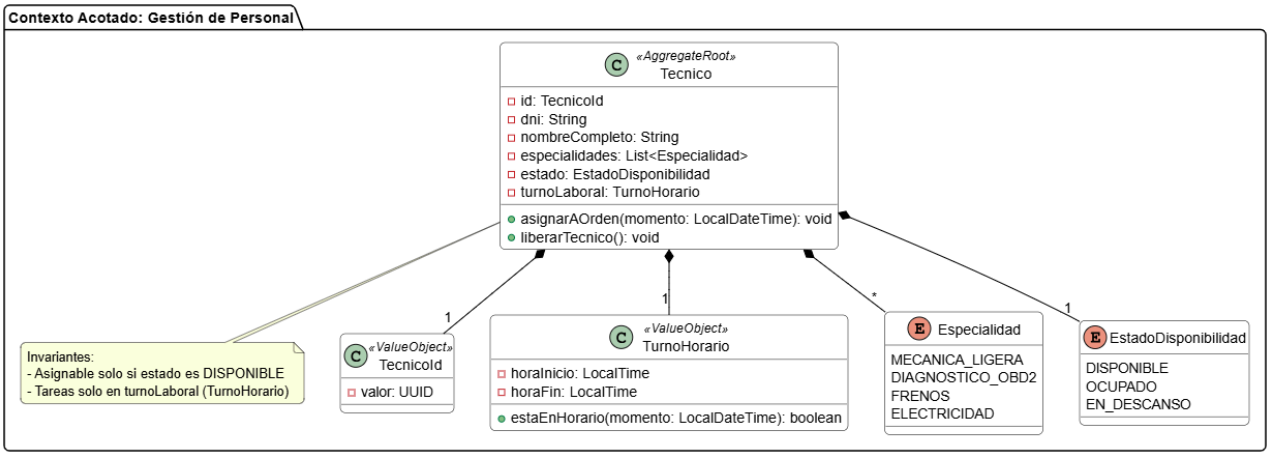
enum EstadoDisponibilidad {
    DISPONIBLE, OCUPADO, EN_DESCANSO
}

// Identificador fuertemente tipado (encapsula UUID)
record TecnicoId(UUID valor) {}

// Objeto de valor para representar el rango horario de trabajo del técnico
record TurnoHorario(LocalTime horaInicio, LocalTime horaFin) {
    /*
     * Regla de validación estricta (Constructor compacto):
     * La 'horaInicio' debe ser cronológicamente anterior (menor) a la
'horaFin'.
     */

    /*
     * Comportamiento del VO:
     * Comprueba si una fecha/hora específica (momento) recae dentro del
rango
     * entre horaInicio y horaFin.
     */
    public boolean estaEnHorario(LocalDateTime momento) { /* ... */ return
false; }
}
```

Diagrama



4.7. CONTEXTO ACOTADO: FACTURACIÓN Y FINANZAS

4.7.1. AGREGADO 13: ComprobantePago

Entidad Raíz

ComprobantePago

Propósito

Registrar y formalizar fiscalmente los cobros generados a partir de los informes de entrega (y órdenes asociadas).

Invariantes / Reglas de Negocio Protegidas

Consistencia en montos: montoTotal siempre debe ser igual a montoSubtotal + montoIgv.
SerieNumero no puede ser nulo en su emisión.
Un comprobante que ya fue enviado a SUNAT (ENVIADO_SUNAT) no puede ser anulado sin especificar un motivo.

Entidades Internas

Nombre	Rol
Ninguna	-

Objetos de Valor Utilizados

VO	Usado por	Compartido
ComprobanteId	Identidad de ComprobantePago	No
InformeEntregaId	Referencia externa	No
Monto	montos (subtotal, igv, total)	Sí

Atributos de la Entidad Raíz

Atributo	Tipo	Obligatorio	Descripción
id	ComprobanteId	Sí	ID interno.
informeEntregaId	InformeEntregaId	Sí	Referencia a qué entrega pertenece.
tipo	TipoComprobante	Sí	FACTURA o BOLETA.
serieNumero	String	Sí	Serie y número del documento.
montoSubtota	Monto	Sí	Base imponible.

Atributo	Tipo	Obligatorio	Descripción
I			
montoIgv	Monto	Sí	Impuesto.
montoTotal	Monto	Sí	Total del comprobante.
estado	EstadoFactura	Sí	Situación actual.
xmlFirmadoUrl	String	No	Ubicación del XML enviado.

Referencias a Otros Agregados

Agregado destino	Mediante	Cardinalidad
InformeEntrega	InformeEntregaId	1:1

Eventos de Dominio Publicados

Evento	Acción que lo dispara
ComprobantePagoEmitido	Al crear un nuevo comprobante.
ComprobantePagoAnulado	Al anular con éxito el comprobante.

Servicios de Dominio Relacionados

Ninguno.

Código

```
/**
 * 1. AGREGADO RAÍZ
 * Gestiona la emisión, anulación y envío a entidades fiscales (SUNAT) de los comprobantes de pago.
 */
public class ComprobantePago {
    // Identificadores y referencias a otros agregados
    private ComprobanteId id;
    private InformeEntregaId informeEntregaId;
```

```
    // Datos del documento fiscal
    private TipoComprobante tipo;
    private String serieNumero;

    // Valores monetarios
    private Monto montoSubtotal;
    private Monto montoIgv;
    private Monto montoTotal;

    // Control de estado y documentos electrónicos
    private EstadoFactura estado;
    private String xmlFirmadoUrl;

    // --- Comportamientos del Agregado ---

    /*
     * Nota de regla de negocio estricta (Constructor / Invariante):
     * El 'montoTotal' debe ser matemáticamente exacto a la suma del
'montoSubtotal' + 'montoIgv'.
     * El comprobante nace por defecto en estado EMITIDO.
     */

    // Asigna la URL del documento XML generado y cambia el estado a
ENVIADO_SUNAT
    public void generarXMLSunat(String url) { /* ... */ }

    /*
     * Regla de negocio estricta:
     * Lanza excepción si el estado actual es ENVIADO_SUNAT y no se
proporciona un motivo válido
     * (es obligatorio justificar la anulación si ya se reportó al fisco).
     * Si pasa la validación, cambia el estado a ANULADO.
     */
    public void anularComprobante(String motivo) throws
IllegalStateException { /* ... */ }
}


/**
 * 2. OBJETOS DE VALOR (Value Objects) Y ENUMS
 * Tipos de datos inmutables y estados permitidos dentro del dominio
contable/fiscal.
 */

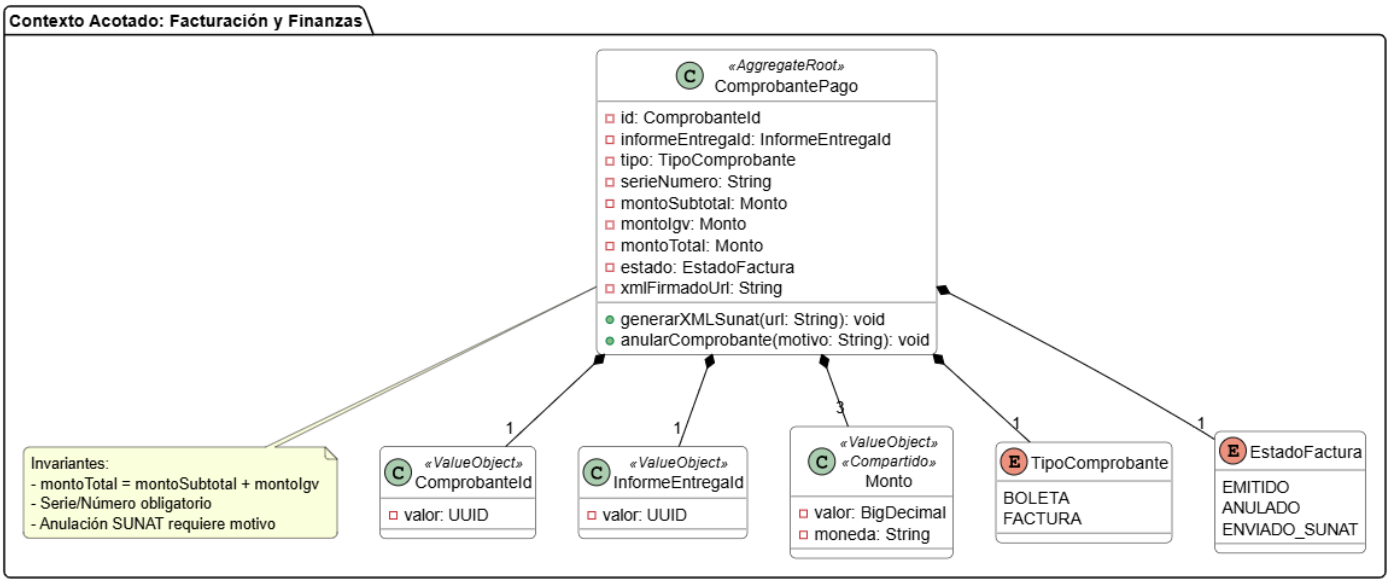
enum TipoComprobante {
    BOLETA, FACTURA
```

```
}

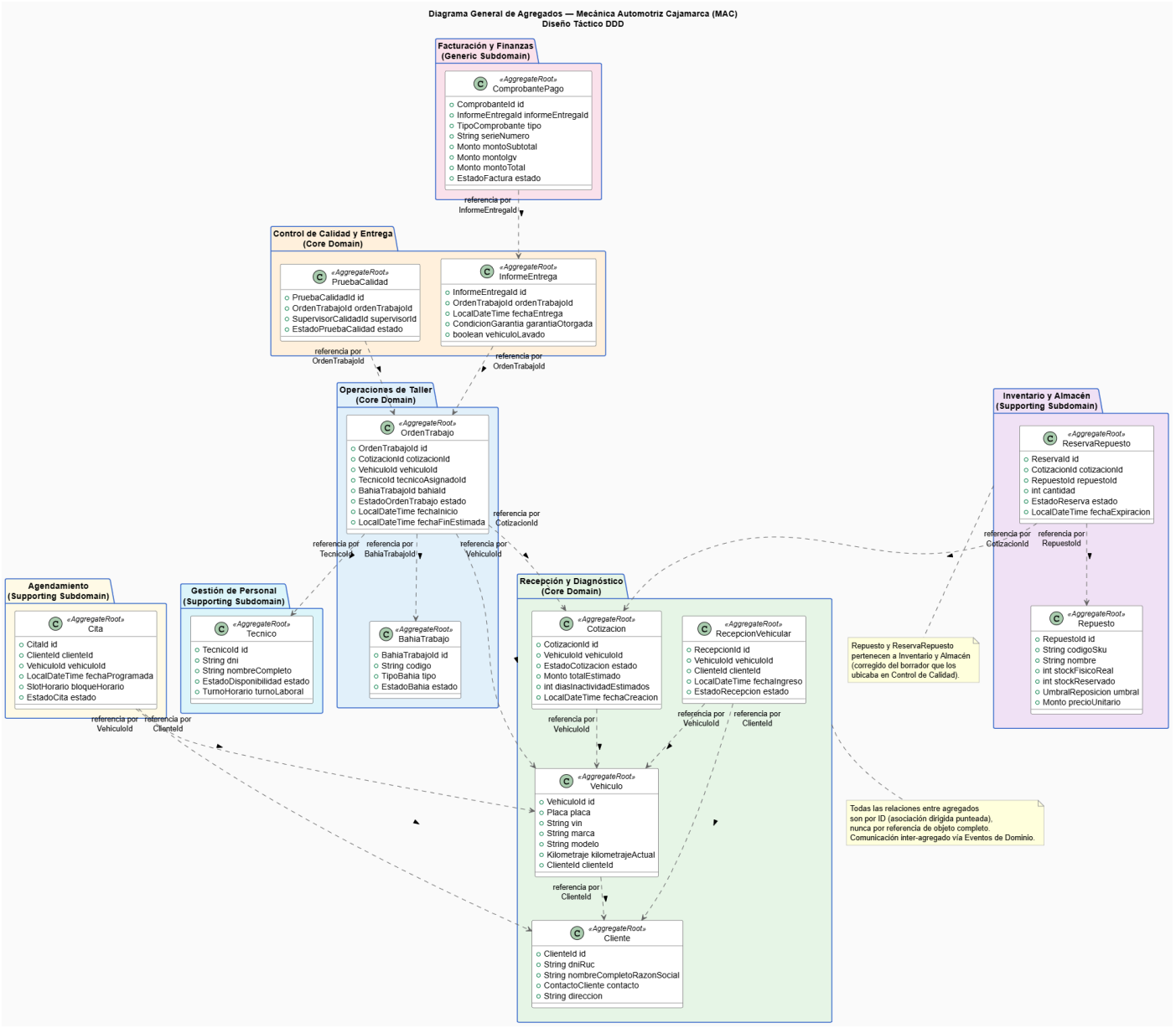
enum EstadoFactura {
    EMITIDO, ANULADO, ENVIADO_SUNAT
}

// Identificadores fuertemente tipados (encapsulan UUID)
record ComprobanteId(UUID valor) {}
record InformeEntregaId(UUID valor) {}
```

Diagrama



4.8. Diagrama de Agregados



5. Servicios de Dominio

Un Servicio de Dominio se emplea para encapsular aquellas operaciones o reglas de negocio complejas que no pertenecen de forma natural a ninguna Entidad o Value Object individual, permitiendo así coordinar múltiples agregados sin alterar su estado de manera inapropiada y manteniendo un modelo de dominio rico en lugar de uno anémico [2], [3].

Para el ecosistema de Mecánica Automotriz Cajamarca (MAC) se han identificado los siguientes Servicios de Dominio estratégicos diseñados específicamente para resolver los cuellos de botella operativos descritos en los apartados anteriores:

5.1. Servicio de Aprobación y Abastecimiento (ServicioAprobacionCotizacion)

Contextos implicados: Recepción y Diagnóstico, Inventario y Almacén
Justificación: Al momento de aprobarse una cotización por parte del cliente resulta insuficiente modificar únicamente el estado del presupuesto, por lo cual este servicio orquesta la creación inmediata de la orden de trabajo junto con la reserva de cada repuesto requerido en el almacén; atacando de esta manera directamente la problemática del "stock fantasma" al garantizar que las piezas queden bloqueadas logísticamente antes de que el vehículo ingrese a las bahías.

```
public class ServicioAprobacionCotizacion {

   public OrdenTrabajo procesarAprobacion(Cotizacion cotizacion, AutorizacionCliente autorizacion,
                                          InventarioRepository inventarioRepo) {
      // Se aplica la regla de negocio de autorización obligatoria exigida por recepción
      cotizacion.aprobar(autorizacion);

      // Se orquesta la reserva inmediata para evitar la desincronización del stock fantasma
      for (ItemCotizacion item : cotizacion.getItems()) {
         if (item.esRepuesto()) {
            Repuesto repuesto = inventarioRepo.buscarPorDescripcion(item.getDescripcion());
            repuesto.reservarStock(item.getCantidad());
         }
      }

      // Se retorna la orden generada lista para ser despachada al taller
      return new OrdenTrabajo(cotizacion.getId(), cotizacion.getVehiculoId());
   }
}
```

5.2. Servicio de Orquestación de Bahías (ServicioAsignacionTaller)

Contextos implicados: Operaciones de Taller, Gestión de Personal
Justificación: La correcta asignación de un vehículo a la zona de trabajo depende de tres variables independientes que deben coincidir simultáneamente, de modo que este servicio centraliza la validación cruzada entre una orden de trabajo en espera, un técnico que cuente con la especialidad adecuada junto con disponibilidad en su turno, y una bahía libre que corresponda al tipo de intervención requerida; asegurando así un flujo continuo sin bloqueos por falta de recursos físicos o humanos en las operaciones diarias.

```
public class ServicioAsignacionTaller {

   public void asignarVehiculoATaller(OrdenTrabajo orden, Tecnico tecnico, BahiaTrabajo bahia) {
```

```
        // Se verifica que el personal técnico asignado cumpla los requisitos de disponibilidad
        if (tecnico.getEstado() != EstadoDisponibilidad.DISPONIBLE) {
            throw new DomainException("El técnico no se encuentra disponible en su turno horario");
        }

        // Se valida que la infraestructura física esté apta para recibir el vehículo
        if (bahia.getEstado() != EstadoBahia.LIBRE) {
            throw new DomainException("La bahía seleccionada se encuentra ocupada o en mantenimiento");
        }

        // Se ejecuta la vinculación de todas las entidades para iniciar el trabajo mecánico
        bahia.ocuparConOrden(orden.getId());
        tecnico.asignarAOrden();
        orden.asignarBahia(bahia.getId());
    }
}
```

5.3. Servicio de Evaluación de Garantías (ServicioEvaluacionGarantia)

Contextos implicados: Calidad y Entrega, Recepción y Diagnóstico
Justificación: Dado que la organización debe cumplir de manera estricta la regla de negocio que otorga una cobertura de tres meses o cinco mil kilómetros para cualquier servicio, este componente asume la responsabilidad de calcular si un automotor que reingresa todavía mantiene este beneficio activo; cruzando para ello la información del kilometraje actual registrado en la entidad del vehículo con las condiciones estipuladas en el informe de entrega de su intervención previa.

```
public class ServicioEvaluacionGarantia {

    public boolean validarCoberturaActiva(Vehiculo vehiculoActual, InformeEntrega informeAnterior,
                                          DateTime fechaReingreso) {
        CondicionGarantia garantia = informeAnterior.getGarantiaOtorgada();

        // Se calcula el tiempo transcurrido desde la entrega validando el umbral de los meses
        long mesesTranscurridos = ChronoUnit.MONTHS.between(informeAnterior.getFechaEntrega(),
fechaReingreso);

        // Se calcula el desgaste físico midiendo la diferencia de kilómetros recorridos
        int kilometrosRecorridos = vehiculoActual.getKilometrajeActual() -
informeAnterior.getKilometrajeAlEntregar();

        // Se retorna el resultado de cruzar ambas métricas con los límites de la garantía
        return mesesTranscurridos <= garantia.getDuracionMeses() &&
               kilometrosRecorridos <= garantia.getKilometrosCobertura();
    }
}
```

6. Eventos de Dominio

Un Domain Event (Evento de Dominio) es un mensaje inmutable que describe algo significativo que ya ocurrió dentro del dominio del negocio. A diferencia de un comando que expresa una intención de hacer algo y puede ser rechazado, un evento representa un hecho consumado que no puede deshacerse.

6.1. Contexto Acotado: Recepción y Diagnóstico

6.1.1. Agregado: Vehiculo

Domain Event	Descripción
KilometrajeActualizado	El vehículo registró un nuevo kilometraje válido, superior al histórico.
DatosPropietarioActualizados	El propietario del vehículo cambió o actualizó sus datos de contacto.

6.1.2. Agregado: RecepcionVehicular

Domain Event	Descripción
InspeccionFisicaRegistrada	Se completó el inventario de carrocería junto al cliente al ingreso.
DiagnosticoOBD2Registrado	El escáner capturó los códigos de falla electrónica del vehículo.
RecepcionVehicularCompletada	El vehículo completó inspección y diagnóstico; listo para cotizar.

6.1.3. Agregado: Cotizacion

Domain Event	Descripción
ItemAgregadoACotizacion	Se añadió una línea de mano de obra o repuesto al presupuesto.
ItemRemovidoDeCotizacion	Se eliminó una línea del presupuesto antes de su aprobación.
CotizacionAprobada	El cliente firmó digitalmente la cotización. Desencadena la creación de la OrdenTrabajo y la ReservaRepuesto.

6.2. Contexto Acotado: Operaciones de Taller

6.2.1. Agregado: OrdenTrabajo

Domain Event	Descripción
OrdenTrabajoCreada	Se generó la instrucción formal de intervención tras la aprobación de la cotización.
BahiaAsignadaAOrden	El vehículo fue trasladado a una bahía específica del taller.

OrdenBloqueadaPorFaltaRepuesto	La orden se bloqueó porque un repuesto figura disponible en sistema pero no existe físicamente (stock fantasma).
TareaReparacionCompletada	Una tarea mecánica específica fue completada con su evidencia fotográfica.
OrdenTrabajoFinalizada	Todas las tareas completadas. Habilita el inicio de la prueba de calidad.

6.2.2. Agregado: BahiaTrabajo

Domain Event	Descripción
BahiaOcupada	La bahía fue asignada a una orden de trabajo activa.
BahiaLiberada	La bahía quedó disponible para recibir un nuevo vehículo.
BahiaPuestaEnMantenimiento	La bahía fue inhabilitada temporalmente por mantenimiento interno.

6.3. Contexto Acotado: Control de Calidad y Entrega

6.3.1. Agregado: PruebaCalidad

Domain Event	Descripción
ResultadosEstaticosRegistrados	El supervisor registró los resultados de luces, fugas y torque.
ResultadosRutaRegistrados	El supervisor registró los resultados de la prueba de ruta.
PruebaCalidadAprobada	El vehículo pasó todas las verificaciones. Habilita la generación del informe de entrega.
PruebaCalidadRechazada	El vehículo no pasó las verificaciones y regresa al taller para retrabajo.

6.3.2. Agregado: InformeEntrega

Domain Event	Descripción
InformeEntregaGenerado	Se emitió el documento final con trabajos, piezas y garantía. Desencadena la facturación.
ProximoMantenimientoProgramado	Se registró el recordatorio predictivo del siguiente mantenimiento.

6.4. Contexto Acotado: Inventario y Almacén

6.4.1. Agregado: Repuesto

Domain Event	Descripción

StockRepuestoAgregado	Ingreso stock físico al almacén tras una orden de compra.
StockRepuestoDeducido	Se consumió stock físico al despachar un repuesto al taller.
UmbralCriticoAlcanzado	El stock disponible bajó del equivalente a 3 días de operación. Dispara orden de compra automática.

6.4.2. Agregado: ReservaRepuesto

Domain Event	Descripción
ReservaRepuestoConfirmada	Se apartó stock para una cotización aprobada, evitando el stock fantasma.
ReservaRepuestoConsumida	El repuesto reservado fue despachado físicamente al taller.
ReservaRepuestoExpirada	La reserva venció sin ser consumida y liberó el stock retenido.

6.5. Contexto Acotado:Agendamiento

6.5.1. Agregado: Cita

Domain Event	Descripción
CitaProgramada	El cliente reservó un turno de atención en el taller.
CitaReprogramada	La cita fue movida a un nuevo slot horario disponible.
CitaCancelada	La cita fue cancelada y el slot horario quedó liberado.

6.6. Contexto Acotado: Gestión de Personal

6.6.1. Agregado: Tecnico

Domain Event	Descripción
TurnoTecnicoActualizado	Se modificó el horario laboral asignado al técnico.
TecnicoEnDescanso	El técnico salió de su turno activo y no está disponible para recibir tareas.
TecnicoDisponible	El técnico está dentro de su turno y listo para recibir tareas de reparación.

6.7. Contexto Acotado: Facturación y Finanzas

6.7.1. Agregado: ComprobantePago

Domain Event	Descripción
ComprobantePagoEmitido	Se generó el documento tributario electrónico conforme a SUNAT.

ComprobantePagoAnulado	El comprobante fue anulado por error o devolución del servicio.

7. Conclusiones

La aplicación rigurosa de los patrones tácticos de DDD (entidades, objetos de valor, agregados, servicios de dominio y eventos de dominio) permitió construir un modelo que refleja fielmente las reglas y procesos del negocio de mecánica automotriz. Cada agregado encapsula sus propias invariantes y expone comportamiento de negocio a través de métodos con significado semántico, evitando la proliferación de lógica dispersa en capas de aplicación o servicios externos.

Mediante el diseño del agregado ReservaRepuesto y el servicio de dominio ServicioAprobacionCotizacion, se implementó un mecanismo de reserva preventiva de repuestos que se activa inmediatamente al aprobarse una cotización. Este enfoque garantiza que el stock lógico del sistema refleje con exactitud la disponibilidad física real, eliminando la desincronización que generaba retrasos operativos y bloqueos en las órdenes de trabajo.

El modelo diseñado cubre la totalidad del flujo operativo, desde el agendamiento de la cita y la recepción del vehículo, pasando por el diagnóstico, cotización, ejecución de reparaciones y control de calidad, hasta la entrega final con garantía y programación de mantenimiento predictivo. Los eventos de dominio documentados aseguran que cada transición de estado significativa quede registrada como un hecho consumado e inmutable, habilitando auditoría y reactividad entre contextos.

La distribución de los 13 agregados en 7 contextos acotados respeta el principio de alta cohesión y bajo acoplamiento. Los contextos *core* (Recepción y Diagnóstico, Operaciones de Taller, Control de Calidad y Entrega) concentran la lógica de negocio diferenciadora, mientras que los subdominios de soporte (Inventario, Agendamiento, Gestión de Personal) y genérico (Facturación) se mantienen desacoplados, facilitando su evolución independiente o eventual sustitución por soluciones externas.

Los tres servicios de dominio identificados (*ServicioAprobacionCotizacion*, *ServicioAsignacionTaller* y *ServicioEvaluacionGarantia*) resuelven operaciones que naturalmente no pertenecen a un único agregado, coordinando múltiples entidades de distintos contextos sin violar los límites de consistencia. Este patrón evita el antipatrón del modelo anémico y mantiene la lógica de negocio dentro de la capa de dominio.

La definición de objetos de valor como Monto, CondicionGarantia, UmbralReposicion, SlotHorario, AutorizacionCliente, entre otros, elimina la dependencia de tipos primitivos para representar conceptos de negocio. Esto fortalece la validación en tiempo de construcción, mejora la legibilidad del código y facilita el mantenimiento del lenguaje ubicuo compartido entre desarrolladores y expertos del dominio.

El nivel de detalle alcanzado en el diseño táctico (con código Java implementado para cada entidad, agregado y servicio de dominio) proporciona una base directamente traducible a la arquitectura de la aplicación. Los eventos de dominio definidos sientan las bases para una futura arquitectura orientada a eventos, permitiendo la comunicación asíncrona entre contextos acotados y la escalabilidad horizontal del sistema.

El modelo garantiza el cumplimiento de reglas de negocio fundamentales como la autorización explícita del cliente mediante firma digital para la aprobación de cotizaciones, la cobertura de garantía de tres meses o cinco mil kilómetros, el control de umbrales de reposición automática de inventario, y la validación de disponibilidad de técnicos y bahías previo a la asignación de órdenes de trabajo.

8. Referencias

[1] Internet Engineering Task Force, "RFC 9562: Universally Unique IDentifiers (UUIDs)," 2024. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc9562.html. [Accedido: 21-ago-2026].

[2] M. Fowler, "DomainDrivenDesign", *MartinFowler.com*, 2015. [En línea]. Disponible en: https://martinfowler.com/bliki/DomainDrivenDesign.html. [Accedido: 22-ago-2026]

[3] C. de la Torre, B. Wagner, y M. Rousos, "Diseño de un modelo de dominio de microservicio", *Microsoft Learn*, 2022. [En línea]. Disponible en: https://learn.microsoft.com/es-es/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/net-core-microservice-domain-model. [Accedido: 22-ago-2026]